CNNs 的优化

讲师：Hao Wang
AI Thrust，信息枢纽

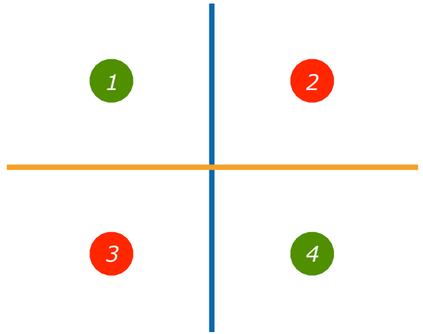
Optimization of CNNs

Instructor: Hao Wang
AI Thrust, information hub

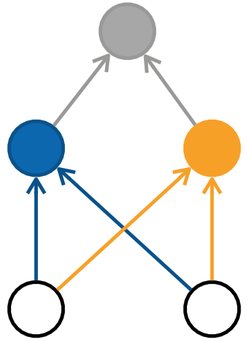
上周回顾

Recap of last week

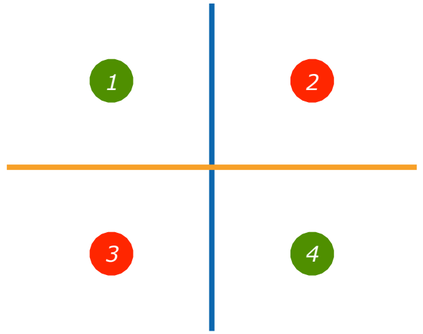
学习 XOR



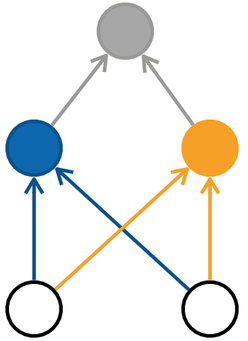
	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+



Learning XOR



	1	2	3	4
	+	-	+	-
	+	+	-	-
product	+	-	-	+



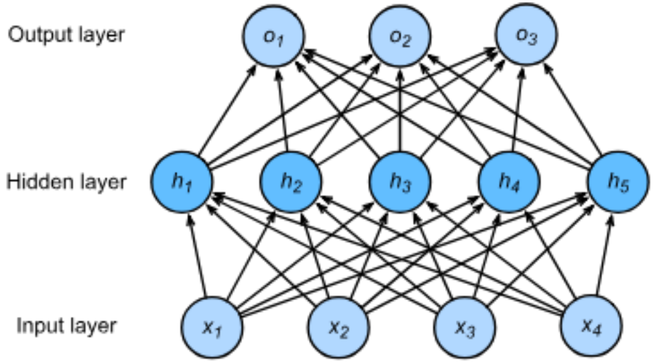
单隐藏层

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{b}_1 \in \mathbb{R}^m$
- 输出 $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}^T \mathbf{h} + b_2$$

σ 是一种逐元素激活函数



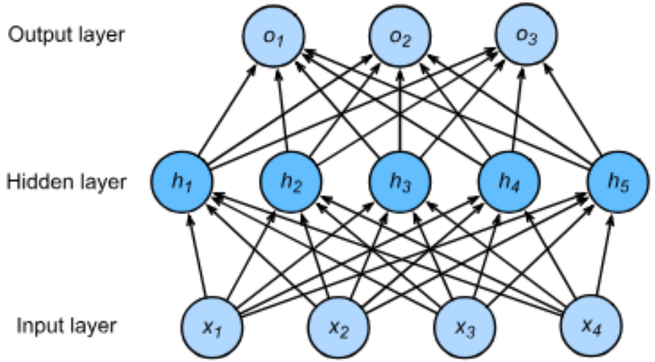
Single Hidden Layer

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T \mathbf{h} + b_2$$

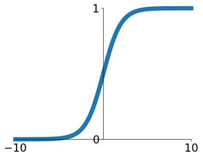
σ is an element-wise activation function



激活函数

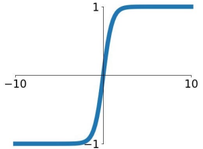
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



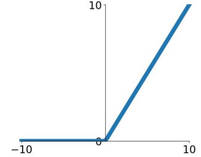
□□□□

$$\tanh(x)$$



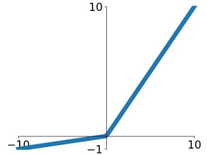
h □a □

$$\max(0, x)$$



泄漏型 ReLU

$$\max(0.1x, x)$$

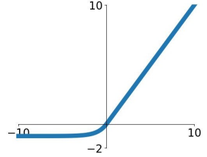


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Ra □

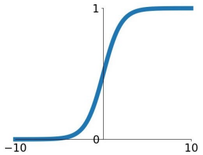
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation Functions

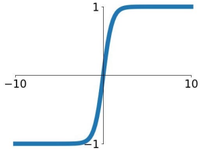
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



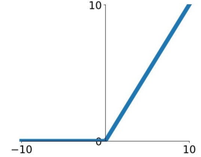
tanh

$$\tanh(x)$$



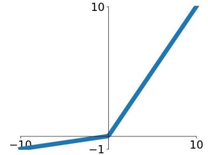
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

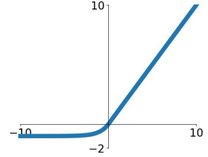


Maxout

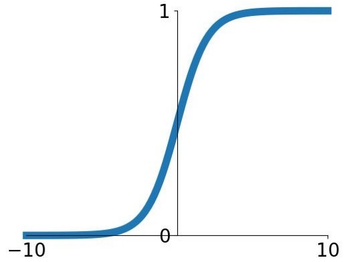
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



激活函数



Sigmoid

$$\sigma(x) = 1/(1+e^{-x})$$

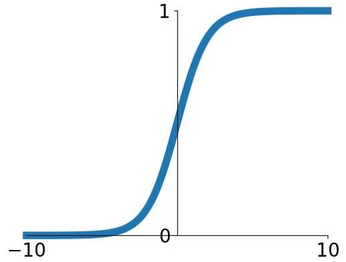
- 将数值压缩到范围 [0,1]
- 历史上很受欢迎，因为它们可以很好地解释为神经元的饱和“放电率”

1. 饱和神经元会“杀死”梯度

2. Sigmoid 输出不是零中心的

为什么这不好?

Activation Functions



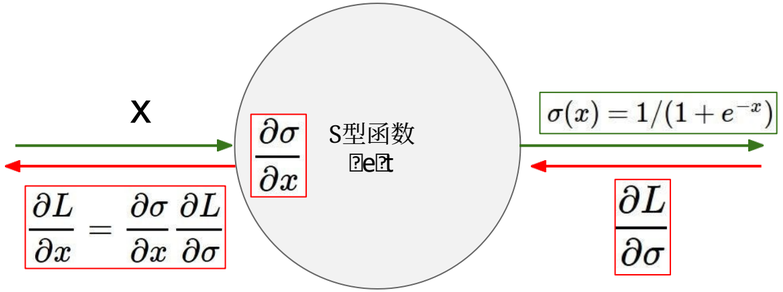
Sigmoid

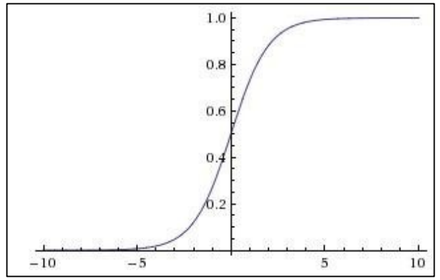
$$\sigma(x) = 1/(1+e^{-x})$$

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered

Why this is bad?

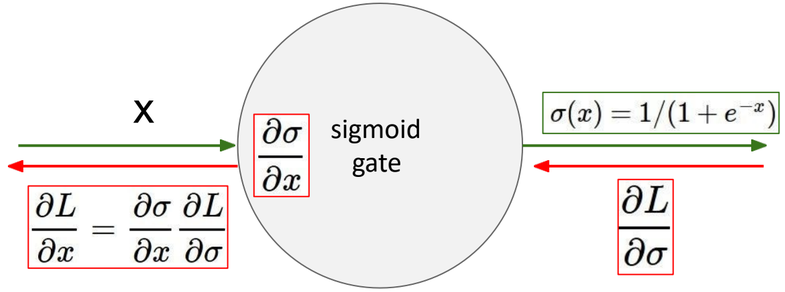


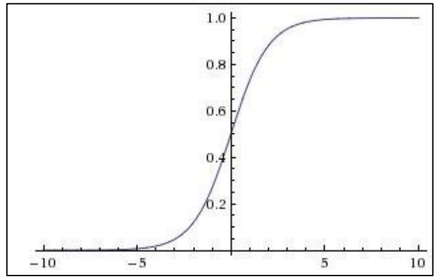


当 x =-10 时会发生什么？ 当 x = 0 时会发生什么？ 当 x = 10 时会发生什么?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

$$\sigma(x) = \sim 1 \qquad \frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$$





What happens when x = -10?
What happens when x = 0?
What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

$$\sigma(x) = \sim 1 \qquad \frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$$

如何解决“饱和神经元”问题?

- 我们需要避免特征值过大或过小

➢ **归一化**!

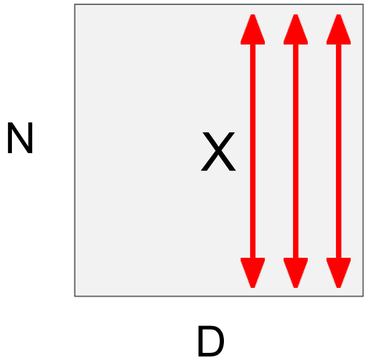
How to resolve the “Saturated Neurons” Problem?

- We need to avoid the feature values too large or too small

➢ **Normalization**!

批归一化

输入v $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ 逐通道均值，形状为 D

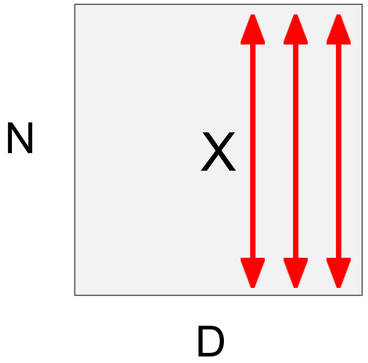
$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ 逐通道方差，形状为 D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ 归一化后的 x，形状为 N x D

问题：如果零均值、单位方差这一约束过于苛刻，该怎么办?

Batch Normalization

Input: $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, Shape is N x D

Problem: What if zero-mean, unit variance is too hard of a constraint?

批量归一化

输入v $x : N \times D$

可学习的缩放和平移参数：

$\gamma, \beta : D$

学习 = ，γ σ

β μ 将恢复恒等函数！

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ 逐通道均值，形状为 D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ 归一化后的 x ，形状为 N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ 输出，形状为 N x D

[Ioffe 和 Szegedy， 2015]

Batch Normalization

Input: $x : N \times D$

Learnable scale and shift parameters:

$\gamma, \beta : D$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, Shape is N x D

[Ioffe and Szegedy, 2015]

批量归一化：测试时

估计值依赖于小批量；在测试时无法这样做！

输入v $x : N \times D$

可学习的缩放和平移参数：

$\gamma, \beta : D$

学习 = 、γ σ
β μ 将恢复恒等函数！

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ 逐通道均值，形状为 D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ 逐通道方差，形状为 D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ 归一化 x，形状为 N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ 输出，形状为 N x D

Batch Normalization: Test-Time

Estimates depend on minibatch; can't do this at test-time!

Input: $x : N \times D$

Learnable scale and shift parameters:

$\gamma, \beta : D$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, Shape is N x D

批量归一化：测试时

输入v $x : N \times D$

可学习的缩放和平移参数：

$\gamma, \beta : D$

在测试期间，批归一化会变成线性算子！可以与其之前的全连接层或卷积层融合

$\mu_j =$ 训练期间所见数值的（滑动）平均值　逐通道均值，形状为 D

$\sigma_j^2 =$ 训练期间所见数值的（滑动）平均值　逐通道方差，形状为 D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

归一化后的 x，形状为 N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

输出，形状为 N x D

Batch Normalization: Test-Time

Input: $x : N \times D$

Learnable scale and shift parameters:

$\gamma, \beta : D$

During testing batchnorm becomes a linear operator! Can be fused with the previous fully-connected or conv layer

$\mu_j =$ (Running) average of values seen during training　Per-channel mean, shape is D

$\sigma_j^2 =$ (Running) average of values seen during training　Per-channel var, shape is D

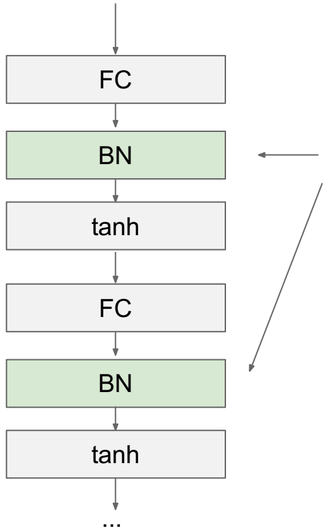
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

批量归一化

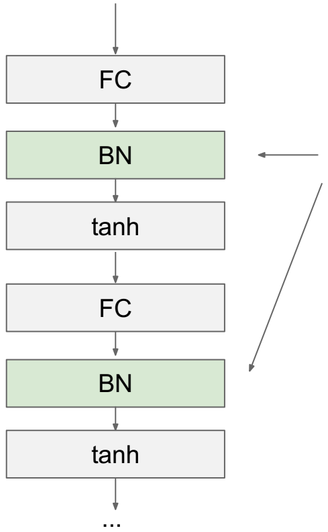


通常插入在全连接层或卷积层之后、非线性激活之前。

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

[Ioffe 和 Szegedy， 2015]

Batch Normalization



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

[Ioffe and Szegedy, 2015]

我们如何确保模型优化良好?

□（上周）非饱和激活函数

□架构

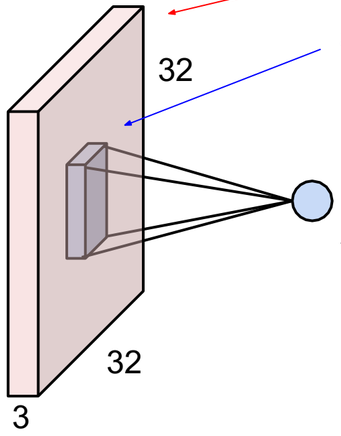
□优化器

□正则化

How do we ensure the model to optimize well?

- (Last week) Non-saturation activation function
- Architecture
- Optimizer
- Regularization

卷积层



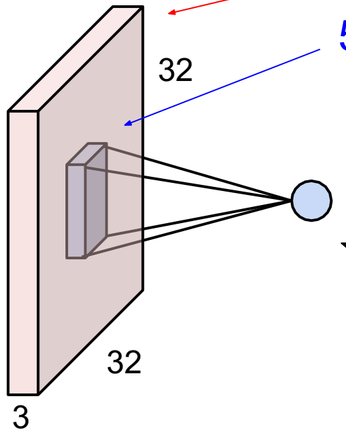
32x32x3 图像

5x5x3 滤波器w

1 个数字：
将滤波器与图像中一个小的 5x5x3 区块进行点积的结果（即 5*5*3 = 75 维点积 +偏置）

$$w^T x + b$$

Convolution Layer



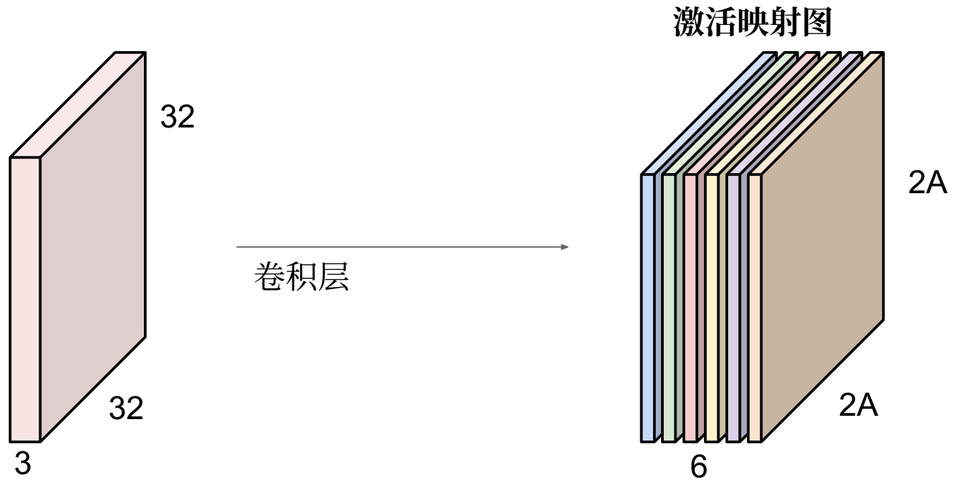
32x32x3 image

5x5x3 filter w

1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

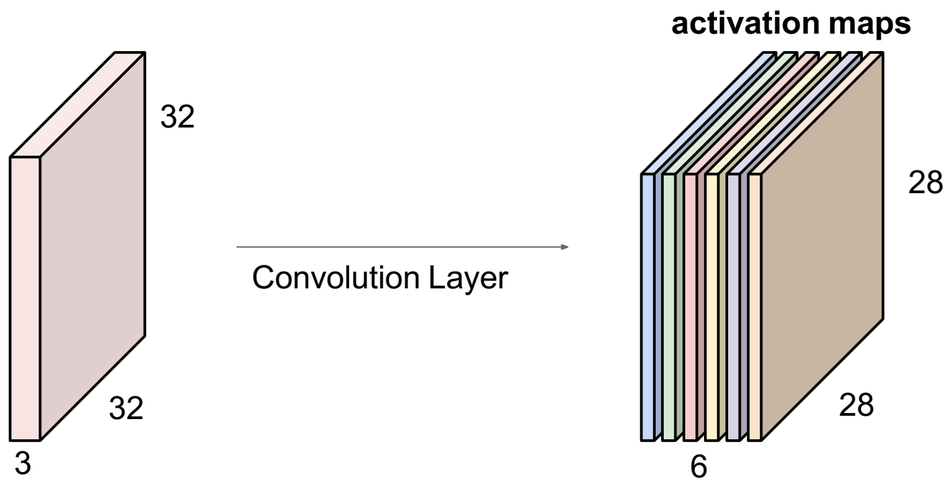
$$w^T x + b$$

例如，如果我们有 6 个 5x5 滤波器，就会得到 6 个单独的激活映射图：



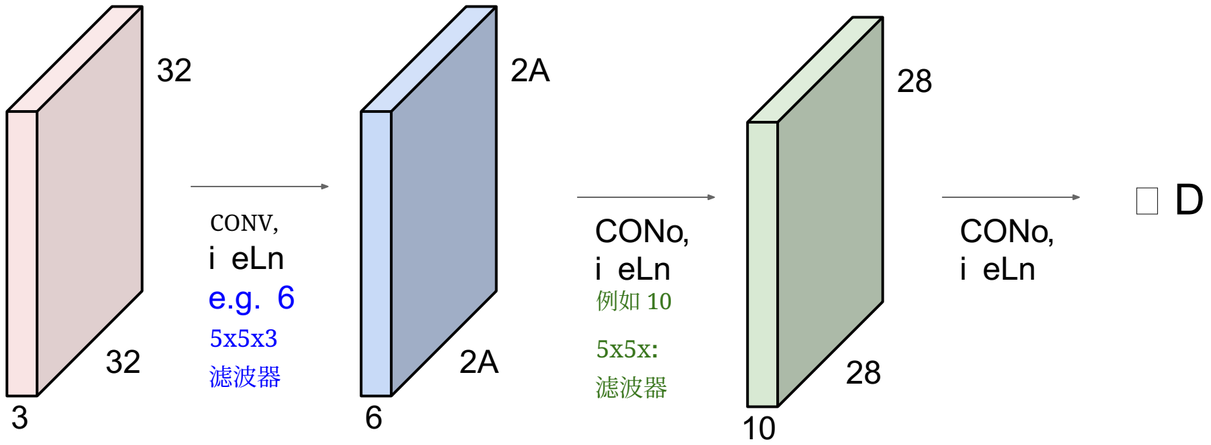
- 我们将这些堆叠起来，得到一张尺寸为 28x28x6 的“新图像”！

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

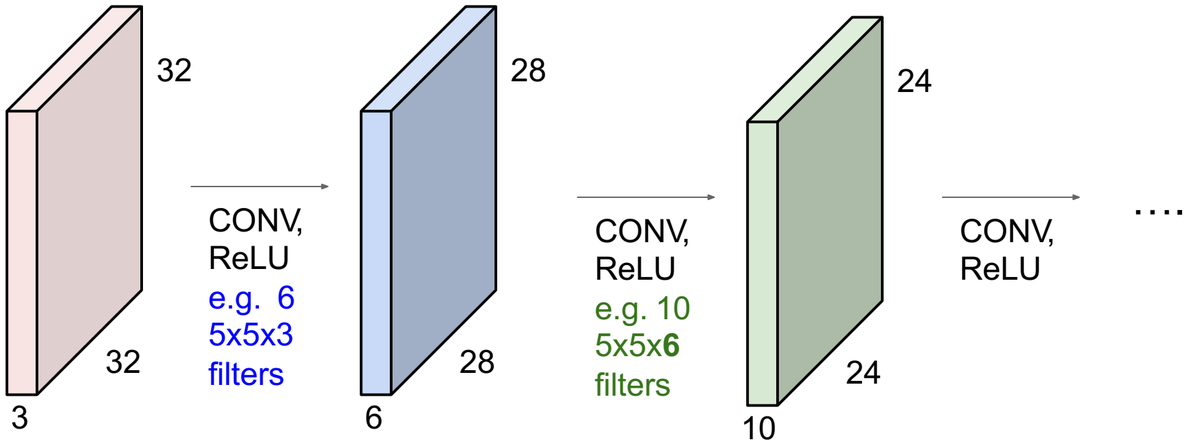


We stack these up to get a “new image” of size 28x28x6!

ConvNet 是一系列卷积层，中间穿插着激活函数



ConvNet is a sequence of Convolution Layers, interspersed with activation functions



我们如何确保模型优化良好?

□（上周）非饱和激活函数

□架构

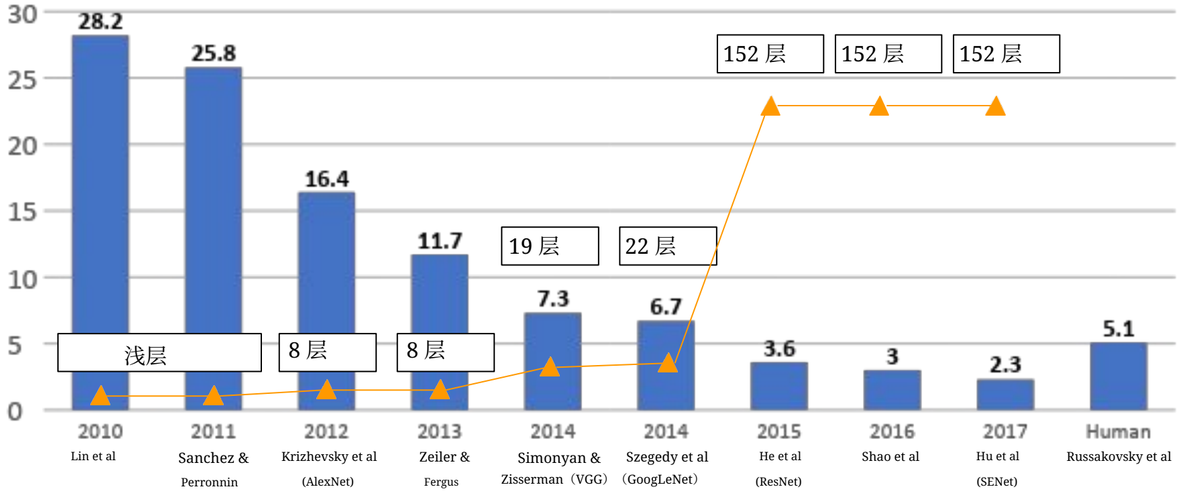
□优化器

□正则化

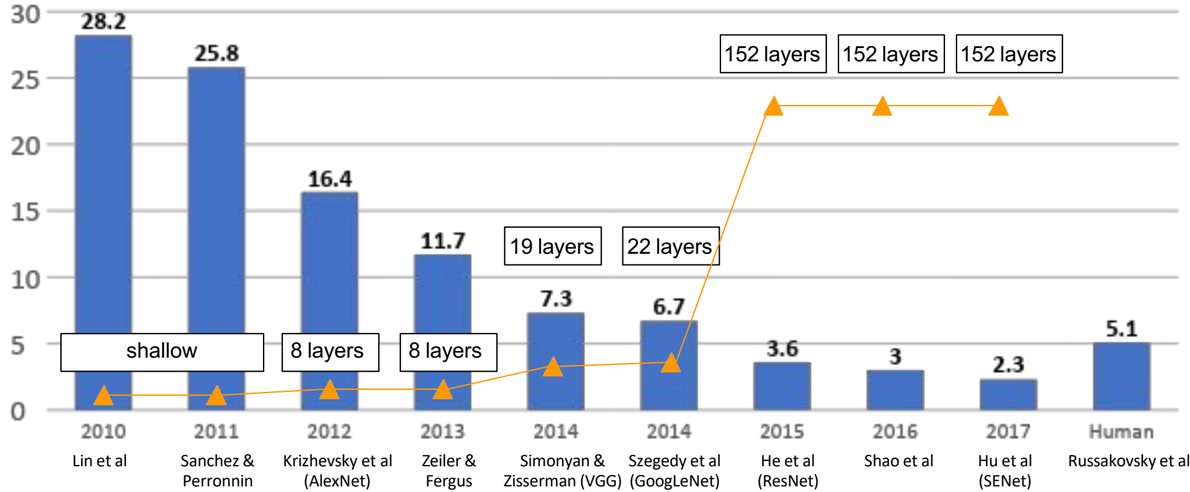
How do we ensure the model to optimize well?

- (Last week) Non-saturation activation function
- Architecture
- Optimizer
- Regularization

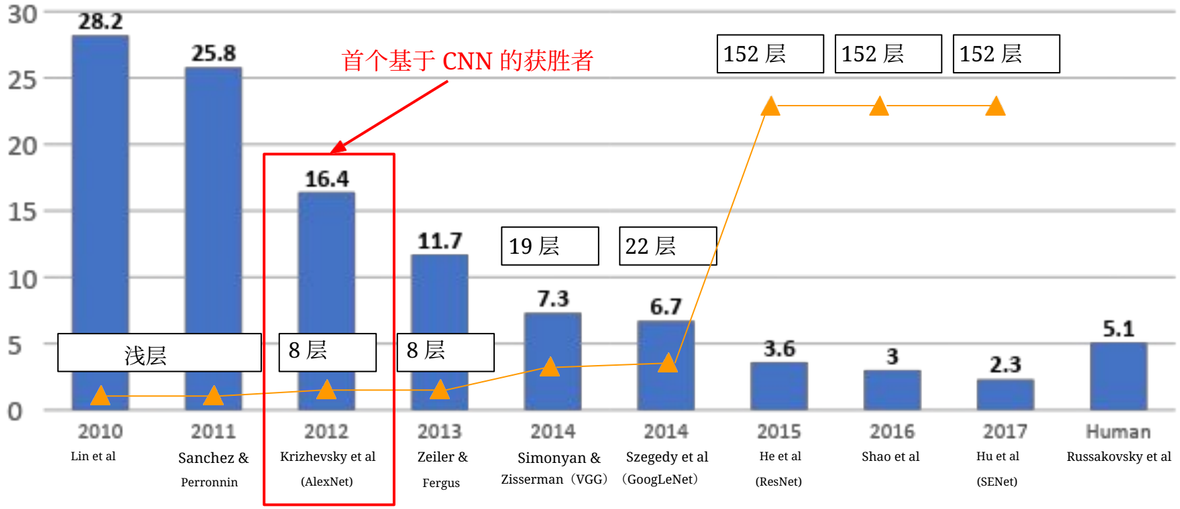
ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



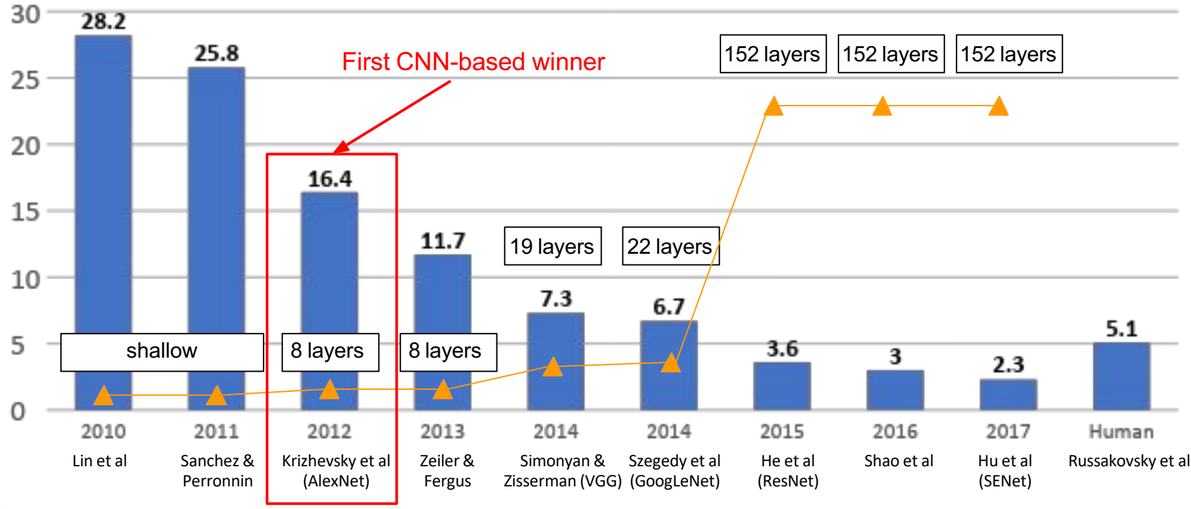
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



ImageNet 大规模视觉识别挑战赛（ILSVRC）获胜者



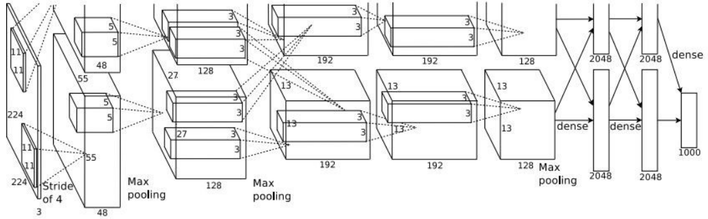
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



案例研究：AlexNet

□*Krizhevsky et al. 5245*□

架构：CONV1MAX
P
O
O
L
1
NORM1CONV2MAX
P
O
O
L
2
N
O
R
M
2
C
O
N
V
3C
ONV4CONV7Ma□
POOL3FC8FC9FC=



图片版权归 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 所有，2012。经许可转载。

Case Study: AlexNet

[Krizhevsky et al. 2012]

Architecture:
CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

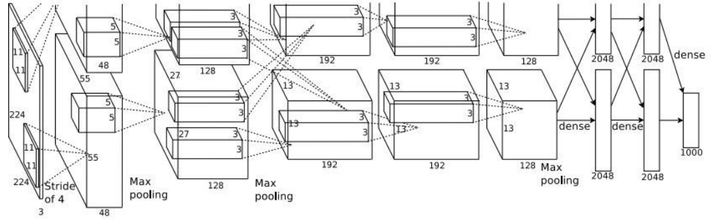
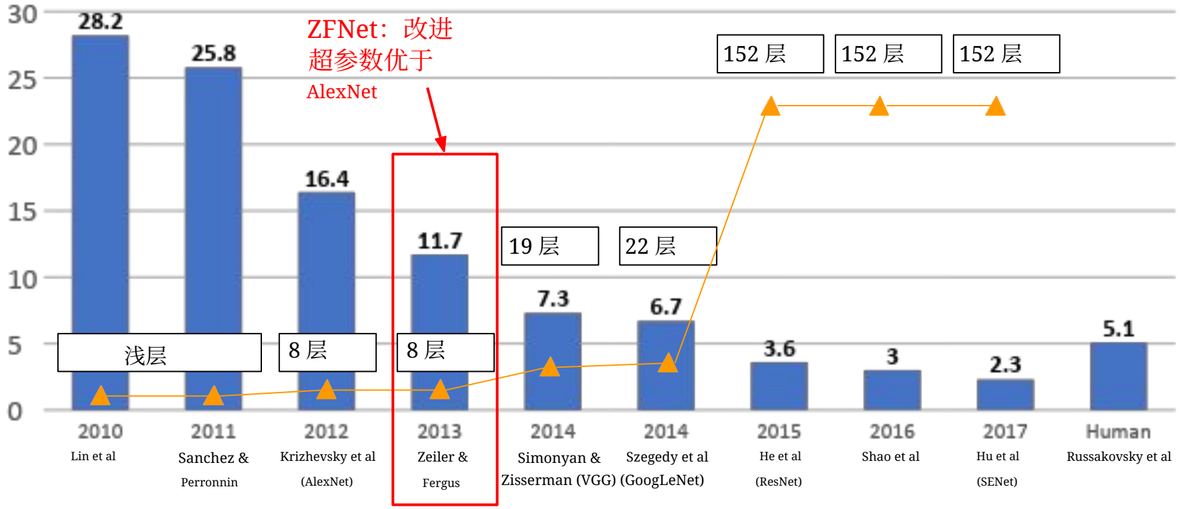
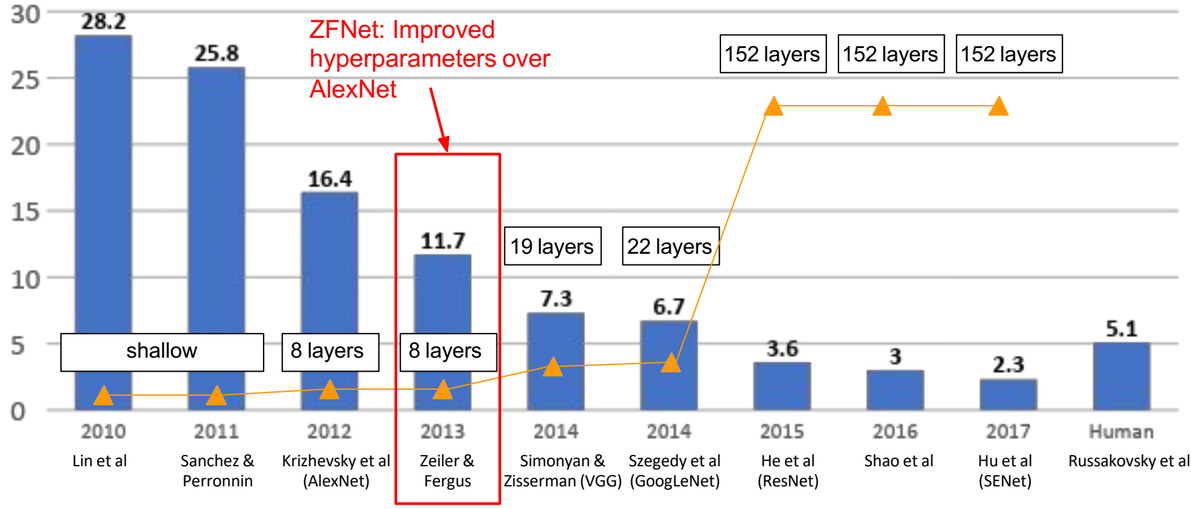


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

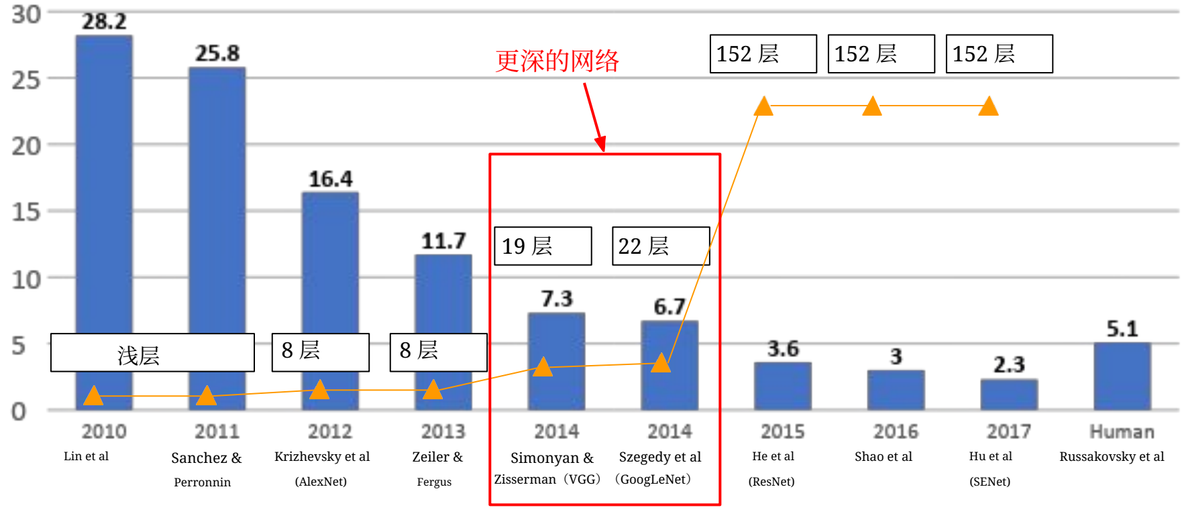
ImageNet 大规模视觉识别挑战赛（ILSVRC）获胜者



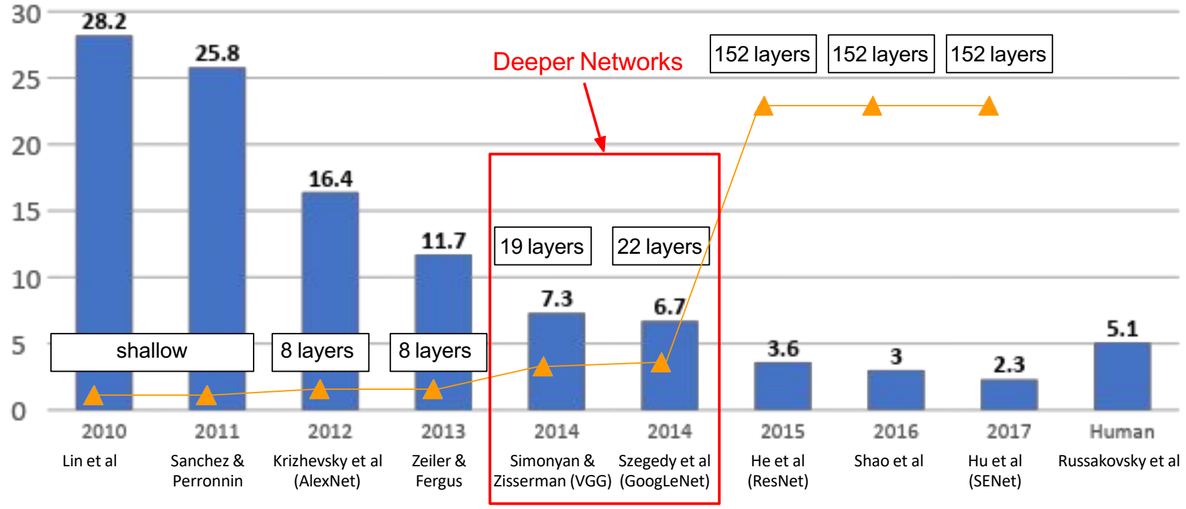
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



案例研究：VGGNet

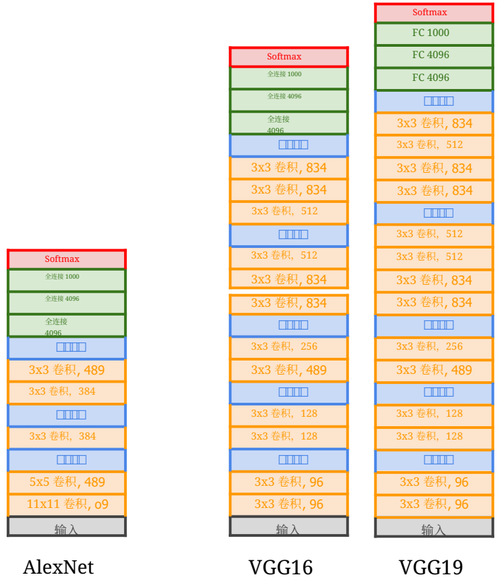
*Simonyan 和 Zisserman， 524[*

问：为什么使用更小的滤波器？（3x3 卷积）

问：三个 3x3 卷积（步幅 1）层的有效感受野是什么？

三个 3x3 卷积（步幅 1）层堆叠起来，
其 **有效感受野** 与一个 7x7 卷积层相同

Wh??



Case Study: VGGNet

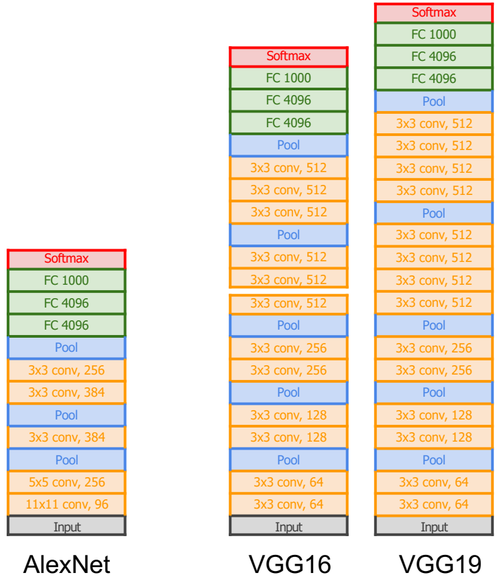
[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

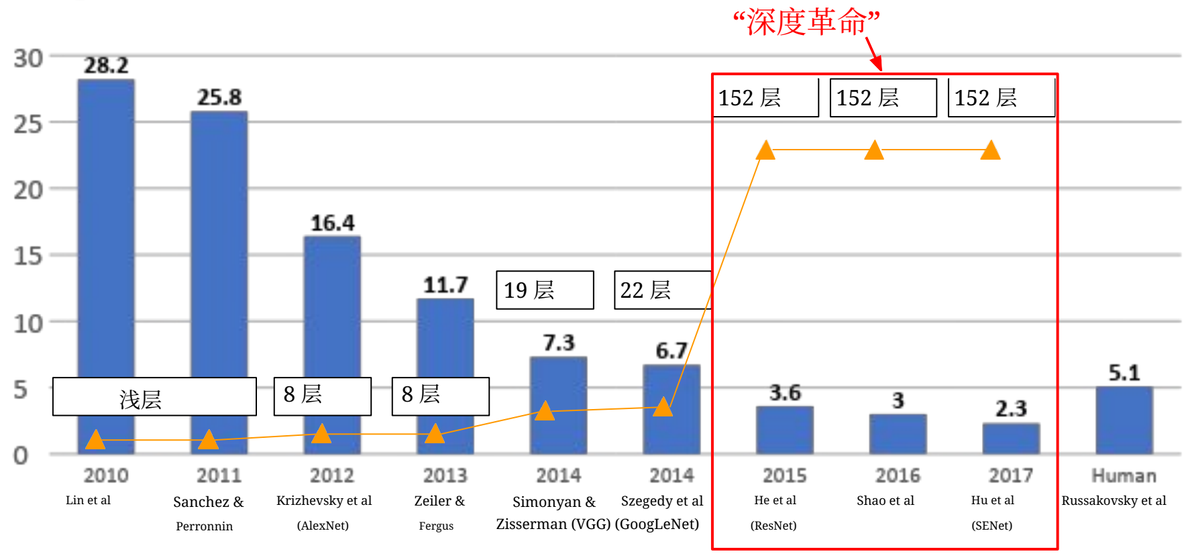
Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

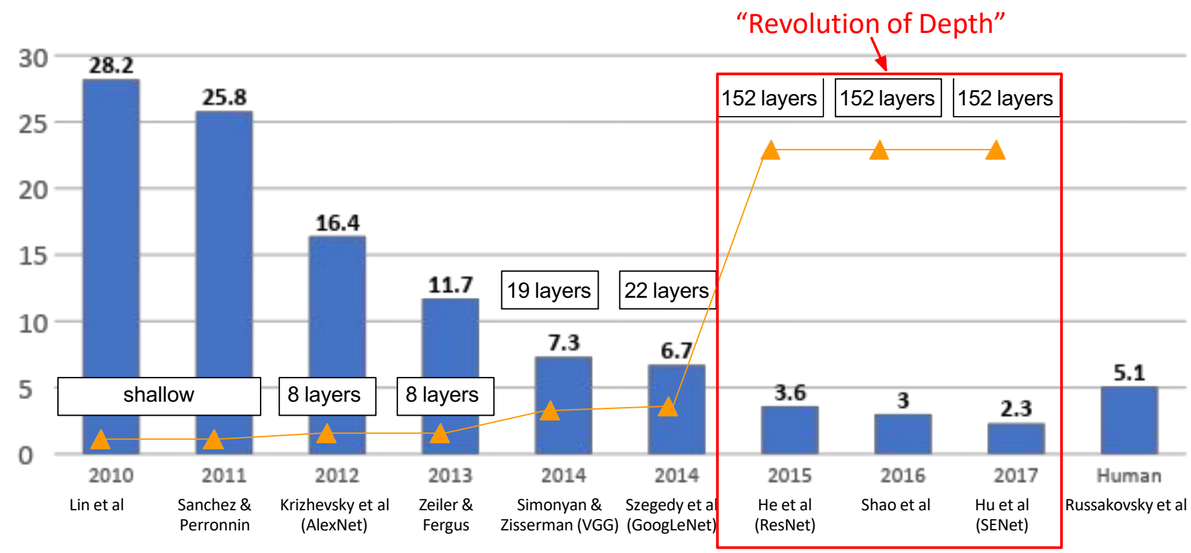
Why?



ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者

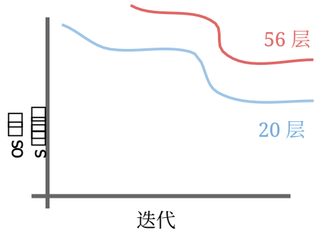


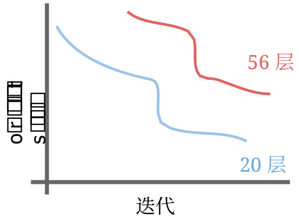
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



如果我们再深入一些会怎样?

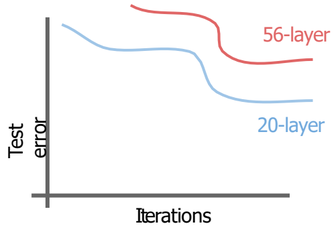
当我们在“普通”卷积神经网络上继续堆叠更深的层时，会发生什么?

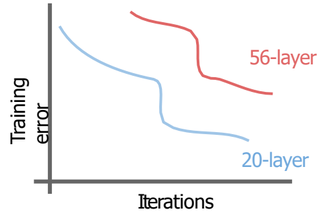




What if we go even deeper?

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

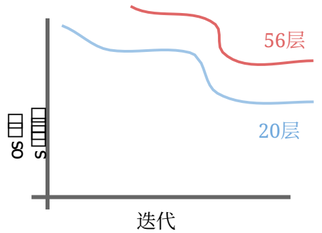


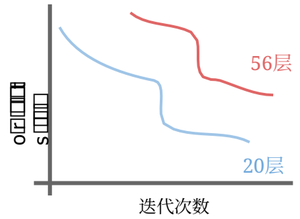


如果我们再深一些呢?

过拟合?

当我们在“普通”卷积神经网络上继续堆叠更深的层时，会发生什么?



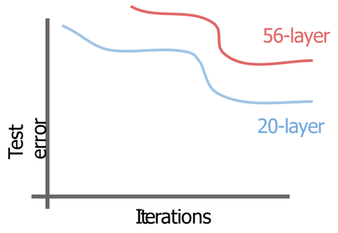


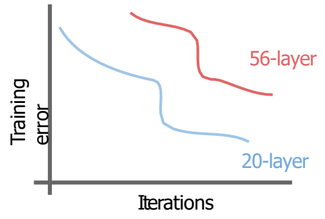
56层模型在测试误差和训练误差上都表现更差 -M更深的模型表现更差，
但这并不是由过拟合导致的！ -M退化问题

What if we go even deeper?

Overfitting?

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



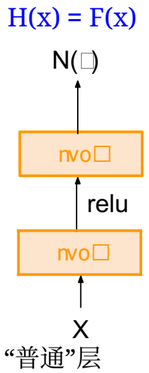


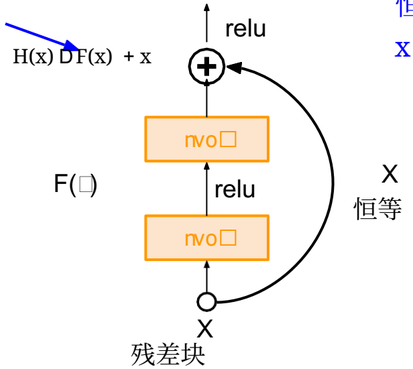
56-layer model performs worse on both test and training error
-> The deeper model performs worse, but it’s not caused by overfitting!
-> Degradation Problem

ResNet

[He et al., 524]

解决方案：使用网络层来拟合残差映射，而不是直接尝试拟合期望的底层映射





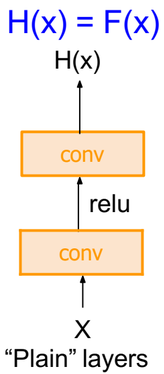
恒等映射：H(x) = x 若 F(x) = 0

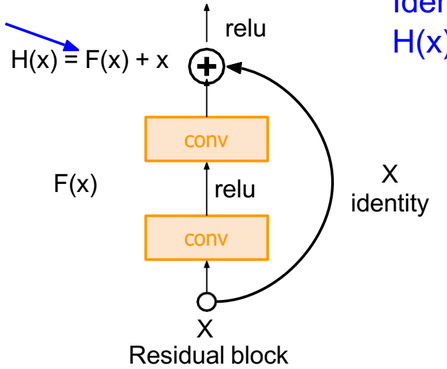
使用层来拟合 残差F(x) = H(x) - x，而不是直接拟合 H(x)

ResNet

[He et al., 2015]

Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping





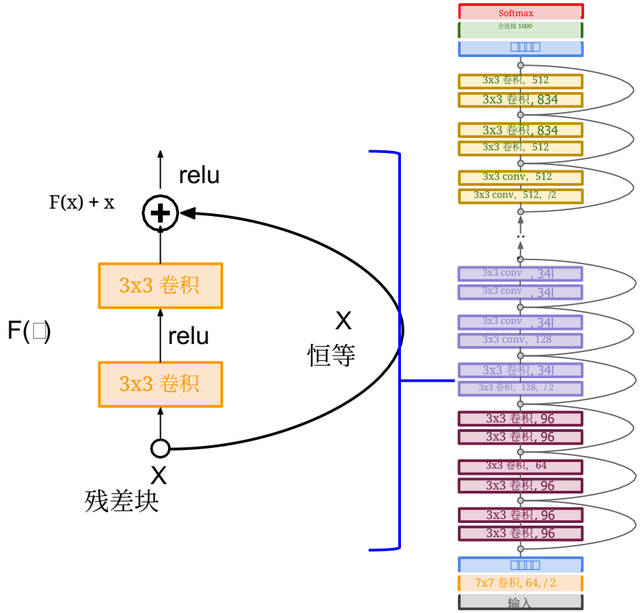
Identity mapping: H(x) = x if F(x) = 0

Use layers to fit **residual** F(x) = H(x) - x instead of H(x) directly

ResNet

[He et al., 524]

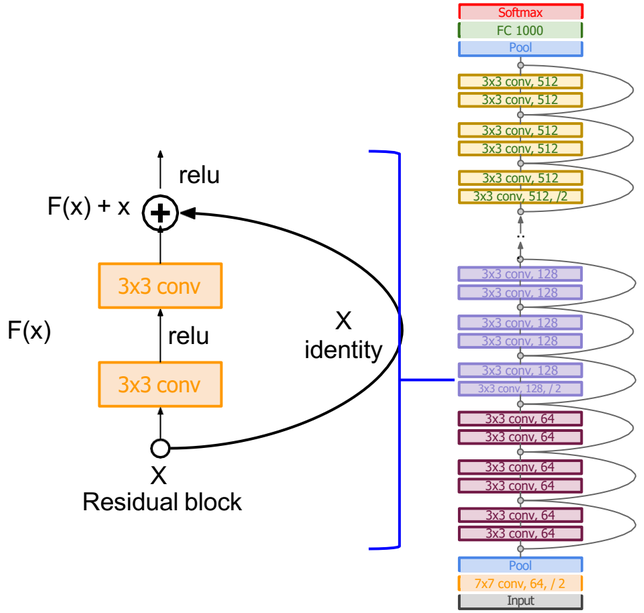
完整 ResNet 架构：
- 堆叠残差块 - 每个残差块有两个 3x3 卷积层



ResNet

[He et al., 2015]

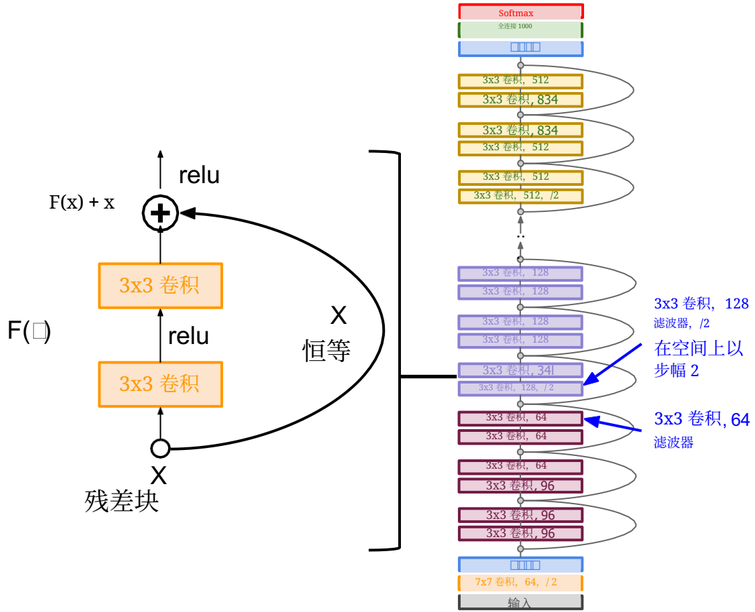
Full ResNet architecture:
- Stack residual blocks
- Every residual block has two 3x3 conv layers



ResNet

□*He et al., 524]* □

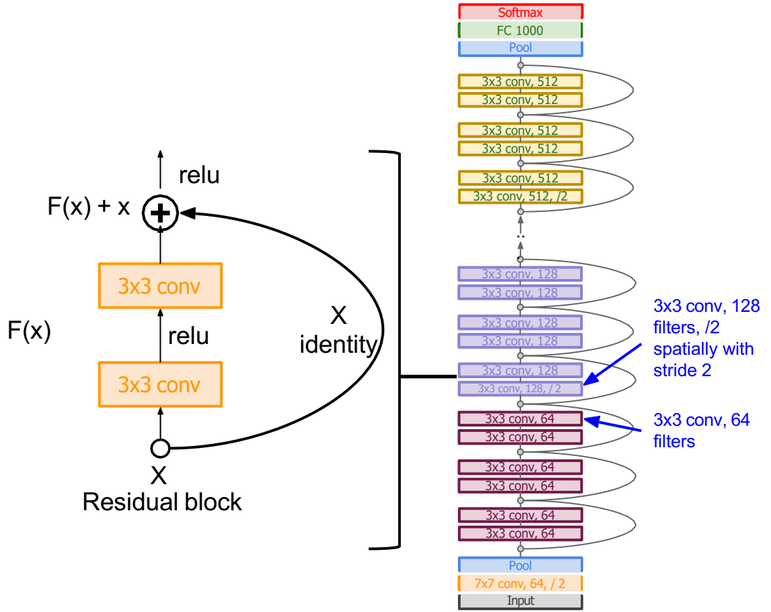
完整 ResNet 架构：
- 堆叠残差块 - 每个残差块都包含两个 3x3 卷积层 - 定期将滤波器数量加倍，并使用步幅 2 在空间上降采样（每个维度 /2）



ResNet

[He et al., 2015]

Full ResNet architecture:
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)

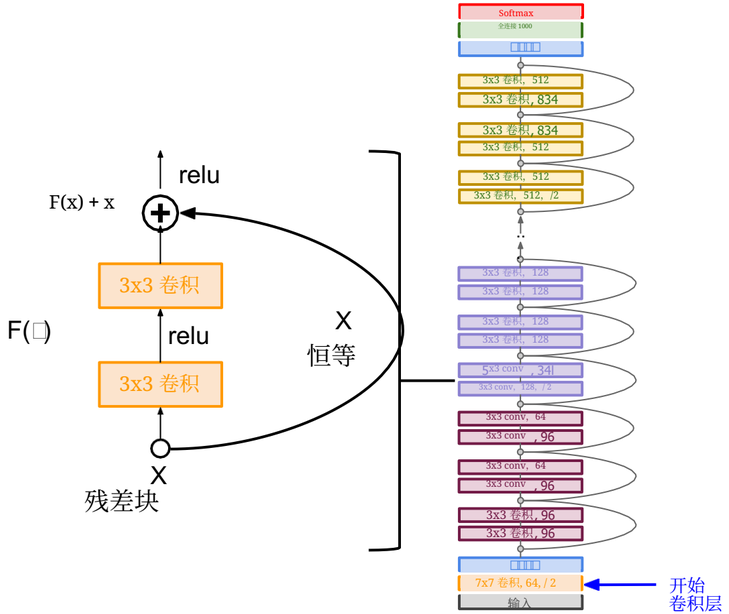


ResNet

[He et al., 524]

完整 ResNet 架构：

- 堆叠残差块 - 每个残差块包含两个 3x3 卷积层 - 定期将滤波器数量加倍，并使用步长 2 进行空间下采样（每个维度 /2） - 在开头增加额外的卷积层（stem）

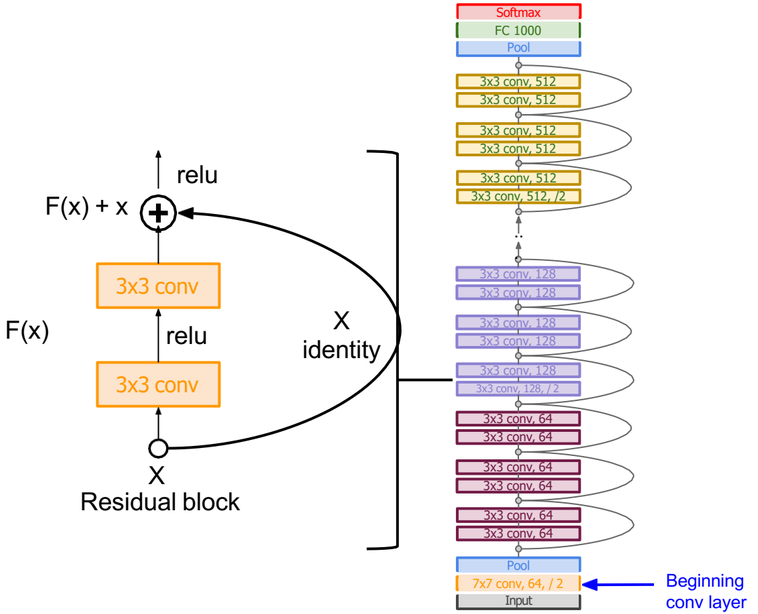


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning (stem)

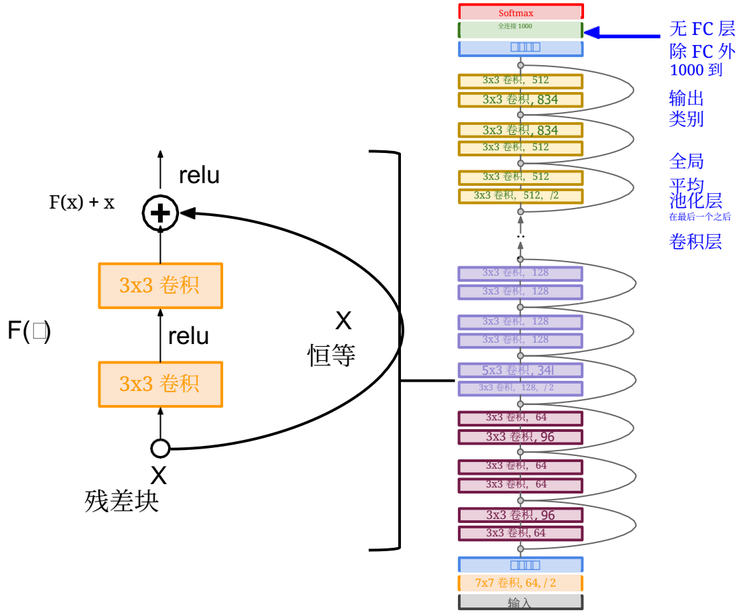


ResNet

□*He et al., 524]* □

完整 ResNet 架构：

- 堆叠残差块 - 每个残差块有两个 3x3 卷积层 - 周期性地将滤波器数量加倍，并使用步幅 2 在空间上降采样（每个维度 /2） - 在开头有一个额外的卷积层（stem） - 末尾没有全连接层（只有 FC 1000 输出类别） - (理论上，你可以训练一个输入图像尺寸可变的 ResNet)

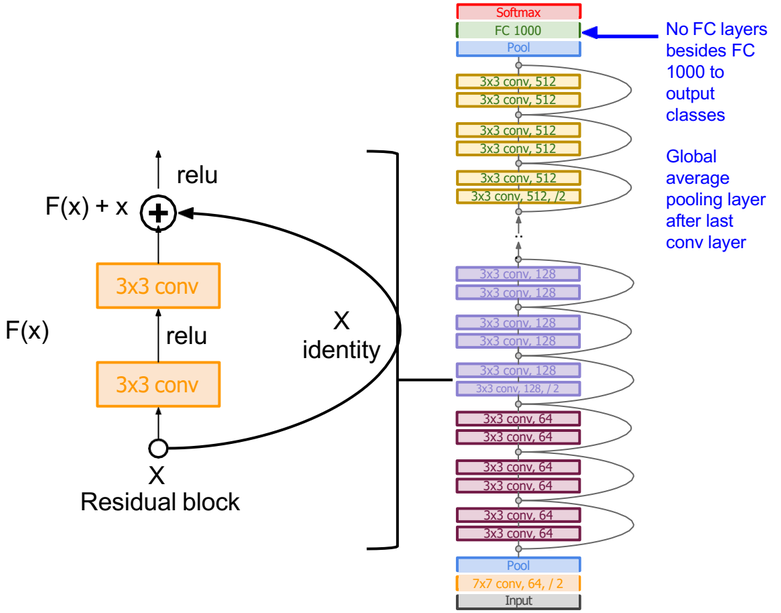


ResNet

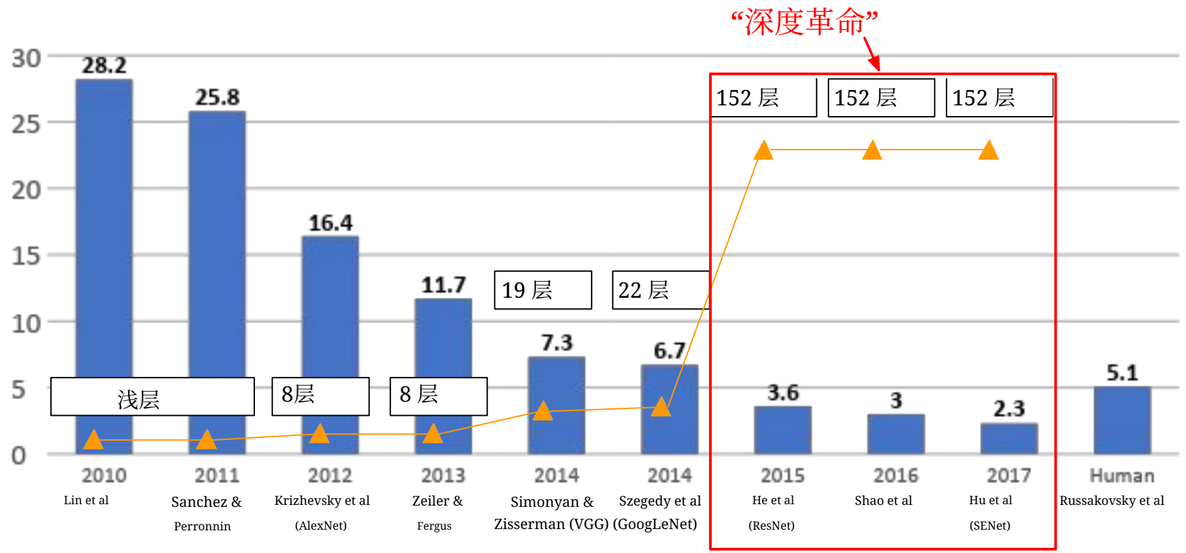
[He et al., 2015]

Full ResNet architecture:

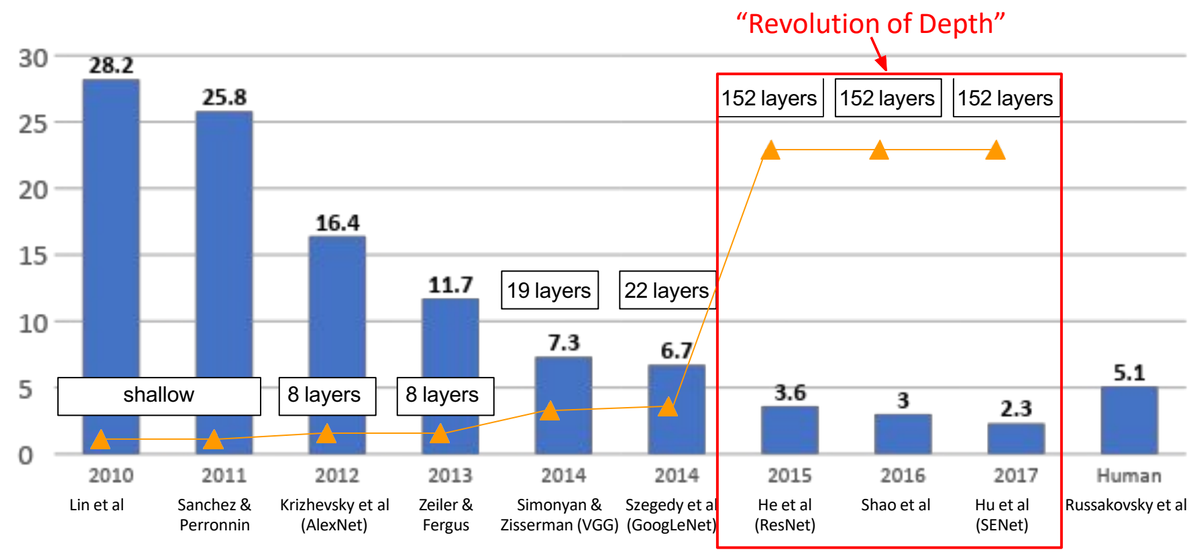
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning (stem)
- No FC layers at the end (only FC 1000 to output classes)
- (In theory, you can train a ResNet with input image of variable sizes)



ImageNet大规模视觉识别挑战赛（ILSVRC）获胜者



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



案例研究：ResNet

He et al., 524]

实验结果
- 能够训练极深网络而不会退化（ImageNet 上 152 层）
- 更深的网络如今如预期般实现了更低的训练误差
- 在 ILSVRC 和 COCO 2015 的所有竞赛中均夺得第一名

MSRA @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**
  - ImageNet Classification: "*Ultra-deep*" (quote Yann) 152-layer nets
  - ImageNet Detection: 16% better than 2nd
  - ImageNet Localization: 27% better than 2nd
  - COCO Detection: 11% better than 2nd
  - COCO Segmentation: 12% better than 2nd

Case Study: ResNet

[He et al., 2015]

Experimental Results
- Able to train very deep networks without degrading (152 layers on ImageNet)
- Deeper networks now achieve lower training error as expected
- Swept 1st place in all ILSVRC and COCO 2015 competitions

MSRA @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**
  - ImageNet Classification: "*Ultra-deep*" (quote Yann) 152-layer nets
  - ImageNet Detection: 16% better than 2nd
  - ImageNet Localization: 27% better than 2nd
  - COCO Detection: 11% better than 2nd
  - COCO Segmentation: 12% better than 2nd

我们如何确保模型能够良好优化?

□（上周）非饱和激活函数

□架构

□优化器

□正则化

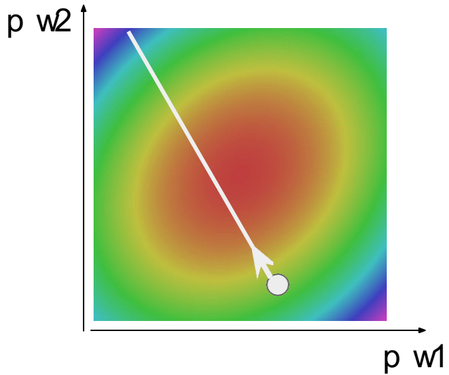
How do we ensure the model to optimize well?

- (Last week) Non-saturation activation function
- Architecture
- Optimizer
- Regularization

前文介绍的优化

```
# Vanilla Gradient Descent

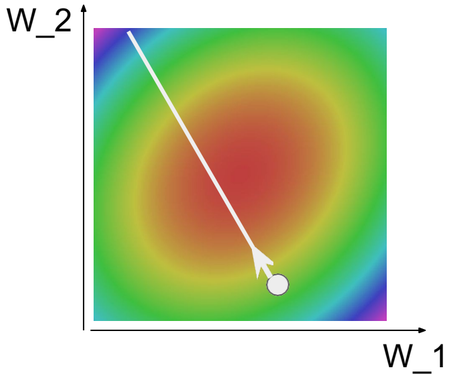
while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```



Optimization introduced previously

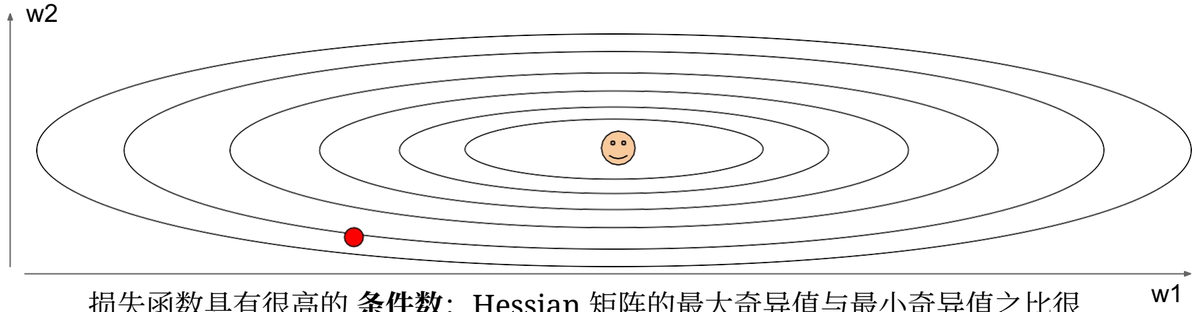
```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```



优化：SGD 的问题 #1

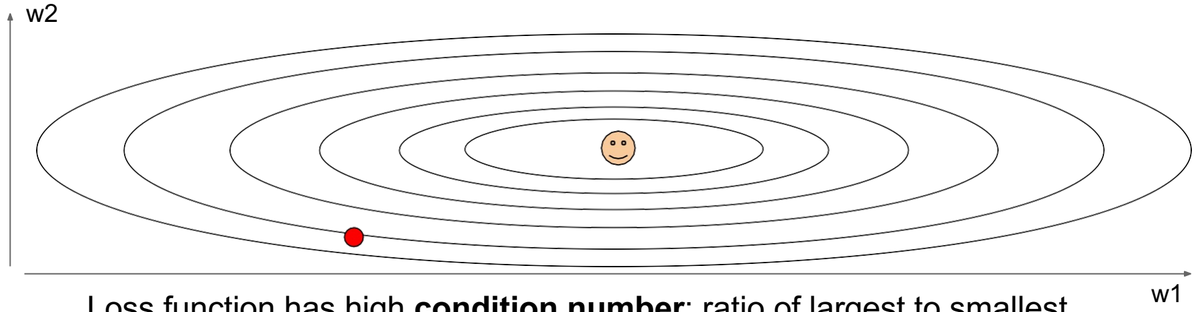
如果损失在一个方向上变化很快，而在另一个方向上变化很慢，会怎样？
梯度下降会怎么做？



损失函数具有很高的 **条件数**：Hessian 矩阵的最大奇异值与最小奇异值之比很大

Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?

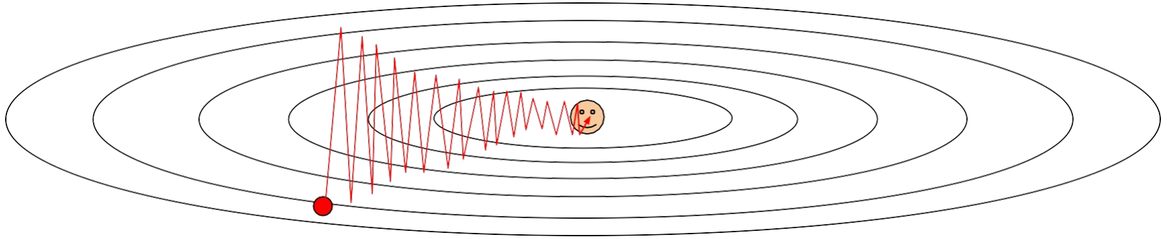


Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

优化：使用 SGD 的问题 #1

如果损失在一个方向上变化很快，而在另一个方向上变化很慢，会怎样？梯度下降会怎么做？ 沿浅维度进展非常缓慢，沿陡峭方向抖动

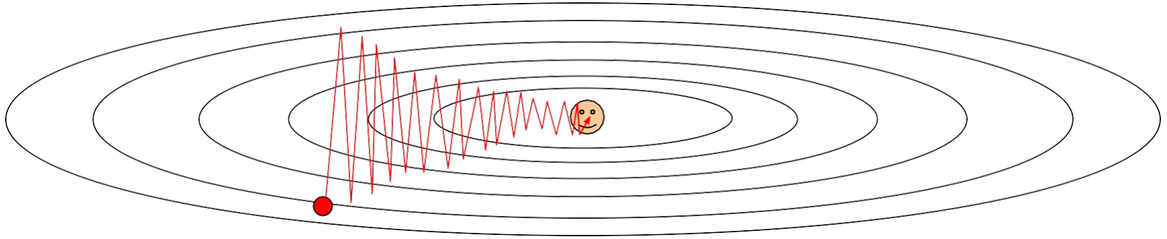
n



损失函数具有很高的**条件数**：Hessian矩阵的最大奇异值与最小奇异值之比很大

Optimization: Problem #1 with SGD

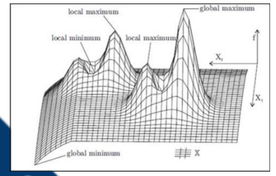
What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter along steep direction

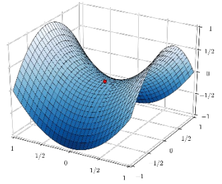


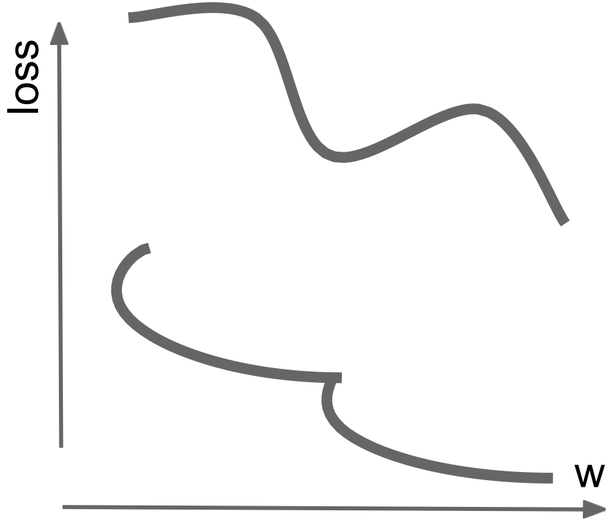
Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

优化：使用 SGD 的问题 #2

如果损失函数存
在**局部极小值**
或**鞍点**会怎样?

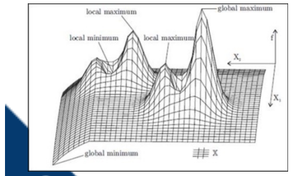


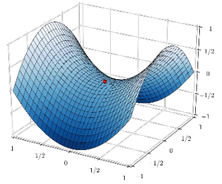


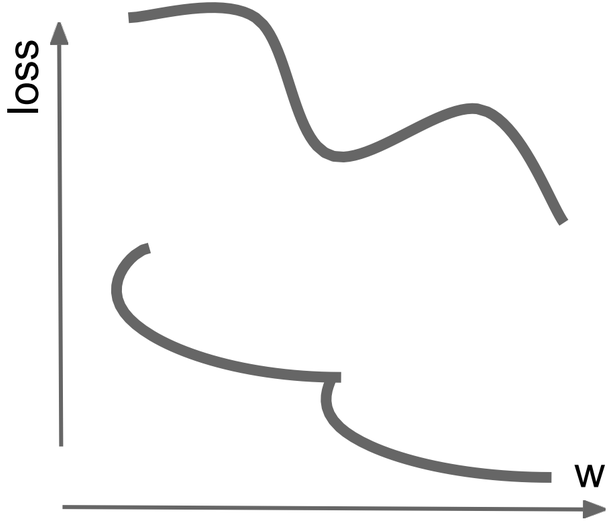


Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?



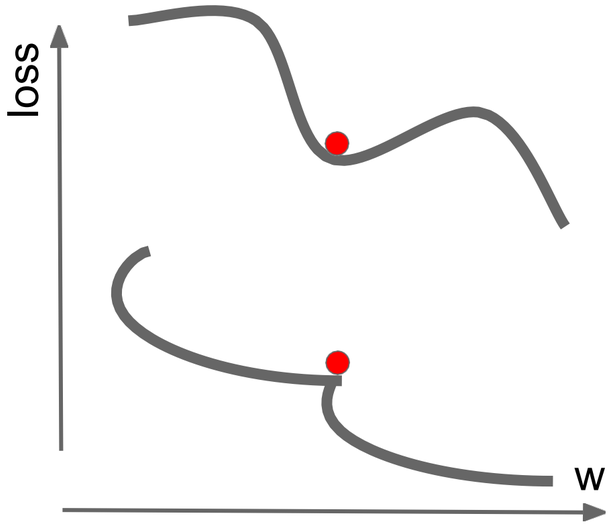




优化：SGD 的问题 #2

如果损失函数存在**局部最小值**或**鞍点**，会怎样?

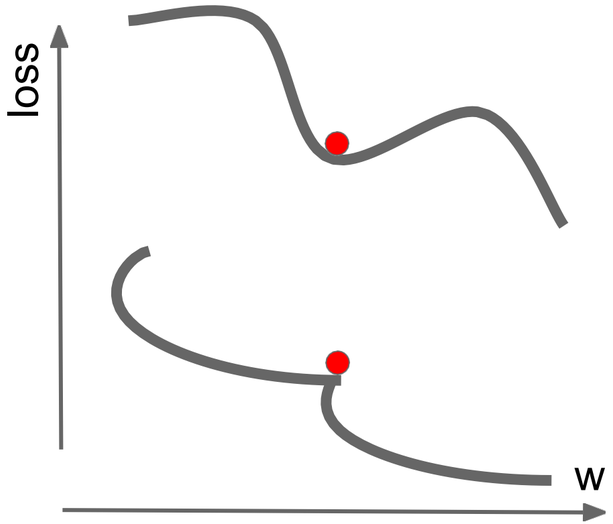
零梯度，梯度下降会陷入停滞



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

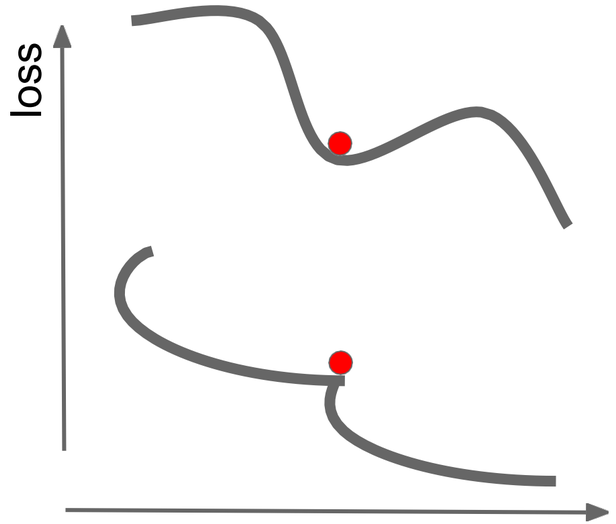
Zero gradient, gradient descent gets stuck



优化：使用 SGD 的问题 #2

如果损失函数存在**局部最小值**或**鞍点**会怎样?

在高维中，鞍点要常见得多

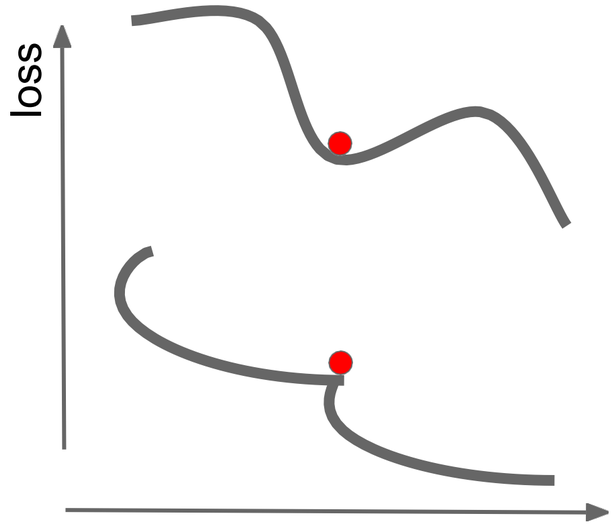


Dauphin 等人，“识别并攻击高维非凸优化中的鞍点问题”，NIPS 2014

Optimization: Problem #2 with SGD

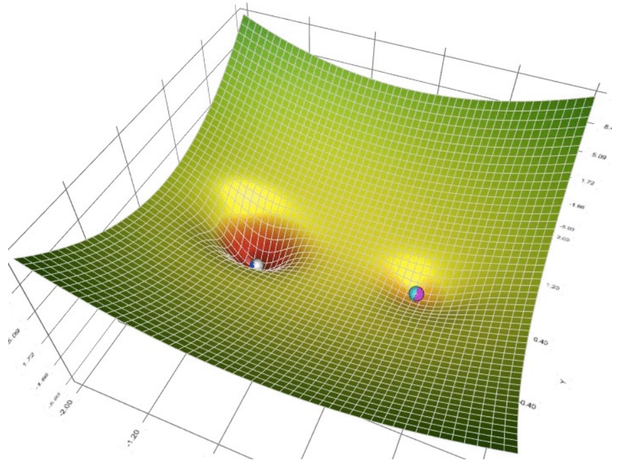
What if the loss function has a **local minima** or **saddle point**?

Saddle points much more common in high dimension

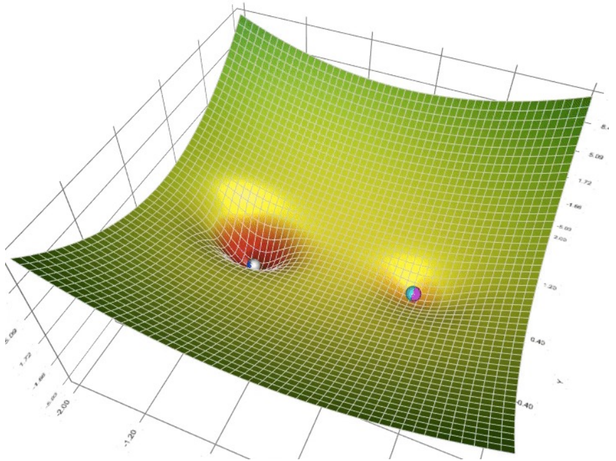


Dauphin et al, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”, NIPS 2014

优化：使用 SGD 的问题 #2



Optimization: Problem #2 with SGD

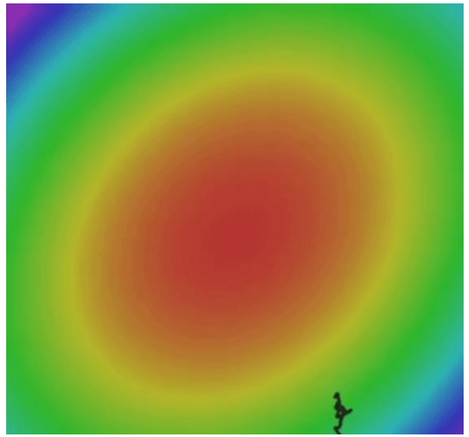


优化：使用 SGD 的问题 #3

我们的梯度来自小批量，因此可能会有噪声！

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$

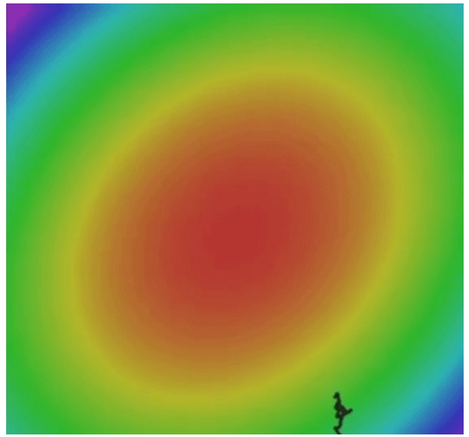


Optimization: Problem #3 with SGD

Our gradients come from minibatches, so they can be noisy!

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$



SGD □ 动量

继续沿与前几次迭代相同的大致方向移动

SGD SGD1动量

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

将“速度”累积为 梯度的滑动平均 Rho 提供“摩擦”；通常 rho=0.9 或 0.99

Sutskever 等，“论深度学习中初始化与动量的重要性”，ICML 2013

SGD + Momentum

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

Build up “velocity” as a running mean of gradients
Rho gives “friction”; typically rho=0.9 or 0.99

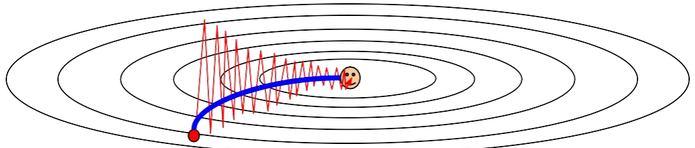
Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

SGD □ 动量

局部极小值　　鞍点

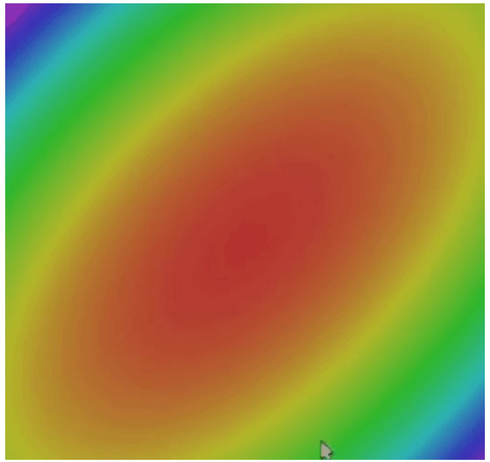


病态条件



局部极小值还是鞍点，哪种情况更可能发生?

htt□s:□□□ □ □ □□outuvy□com□□ atch□□□no□v□□□□a l□



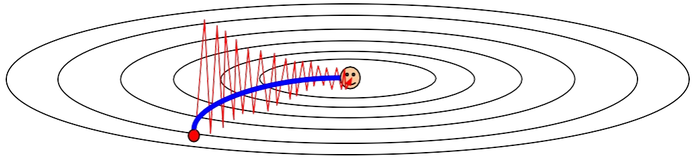
aMH　　SGD+动量

SGD + Momentum

Local Minima　　Saddle points

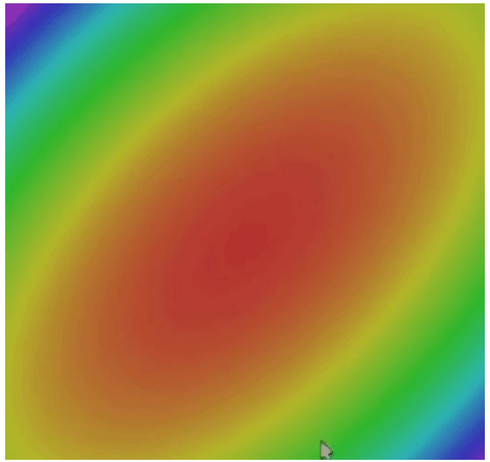


Poor Conditioning



Local minima or saddle point, what is more likely to happen?

https://www.youtube.com/watch?v=noxb_1zpDIw



SGD　　SGD+Momentum

SGD □ 动量

继续朝着与之前迭代相同的总体方向移动SGD SGD1动量

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

- - 将“速度”构建为梯度的运行均值，Rho 提供“摩擦”；通常 rho=0.9 或 0.99

Sutskever et al，“论深度学习中初始化与动量的重要性”，ICML 2013

SGD + Momentum

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

?mS ?o omentum另一种等价表述

SGD1Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t)$$
$$x_{t+1} = x_t + v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx - learning_rate * dx
  x += vx
```

SGD1Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

你可能会看到 SGD+Momentum 以不同方式表述，但它们是等价的——给出相同的 x 序列

Sutskever et al, "On the importance of initialization and momentum in deep learning", ICML 2013

SGD + Momentum

alternative equivalent formulation

SGD+Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t)$$
$$x_{t+1} = x_t + v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx - learning_rate * dx
  x += vx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

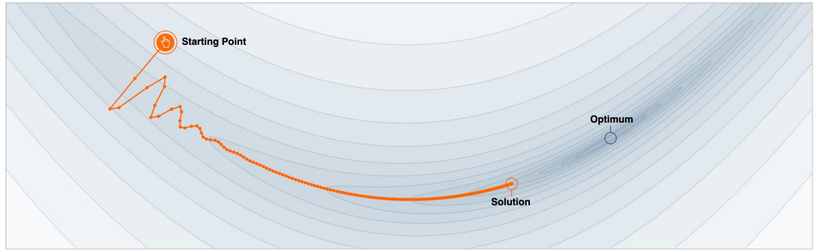
You may see SGD+Momentum formulated different ways, but they are equivalent - give same sequence of x

Sutskever et al, "On the importance of initialization and momentum in deep learning", ICML 2013

SGD □动量

https://distill.pub/2017/momentum/

Why Momentum Really Works



Step-size α = 0.0015

0 0.003 0.006

Momentum β = 0.99

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

GABRIEL GOH
UC Davis

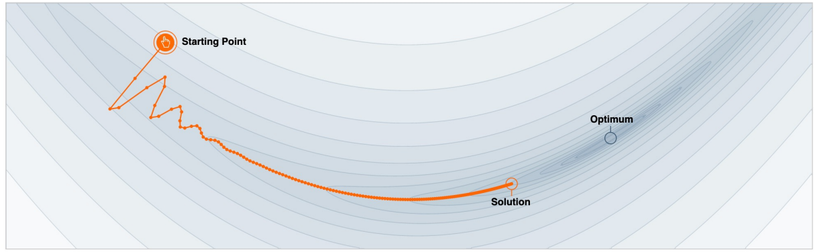
April. 4 2017

Citation: Goh, 2017

SGD + Momentum

https://distill.pub/2017/momentum/

Why Momentum Really Works



Step-size α = 0.0015

0 0.003 0.006

Momentum β = 0.99

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

GABRIEL GOH
UC Davis

April. 4 2017

Citation: Goh, 2017

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

添加了基于每个维度历史平方和的梯度逐元素缩放

“逐参数学习率”或“自适应学习率”

Duchi 等，<用于在线学习和随机优化的自适应次梯度方法>，JMLR 2011

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

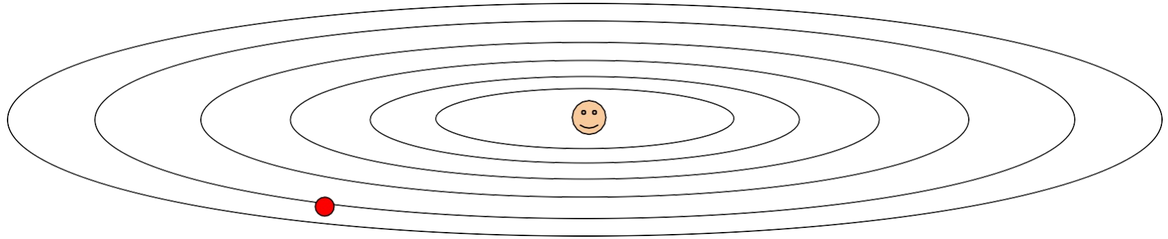
Added element-wise scaling of the gradient based on the historical sum of squares in each dimension

“Per-parameter learning rates”
or “adaptive learning rates”

Duchi et al, “Adaptive subgradient methods for online learning and stochastic optimization”, JMLR 2011

AdaGrad

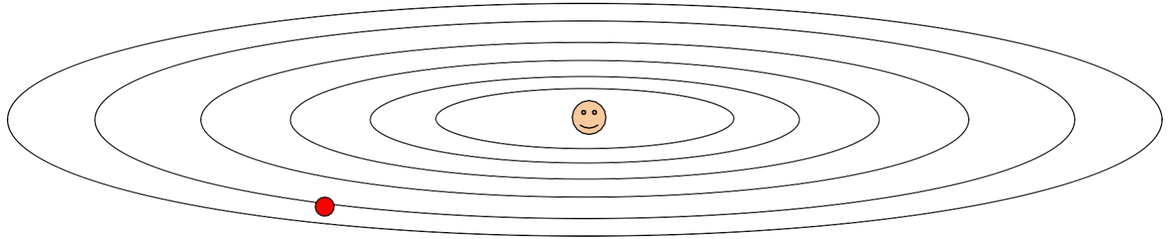
```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



问：AdaGrad 会发生什么?

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

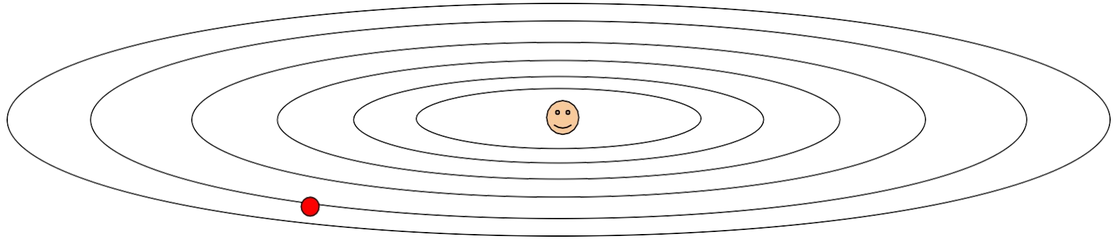


Q: What happens with AdaGrad?

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Wh??



问：AdaGrad 会发生什么？

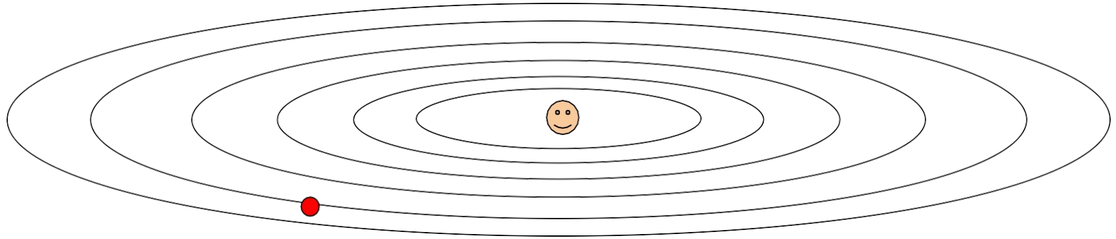
沿“陡峭”方向的进展会被抑制；沿“平坦”方向的进展会被加速

如果 dx 很大，np.sqrt(grad?squared) 会迅速增大，分母将有助于降低学习率

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Why?



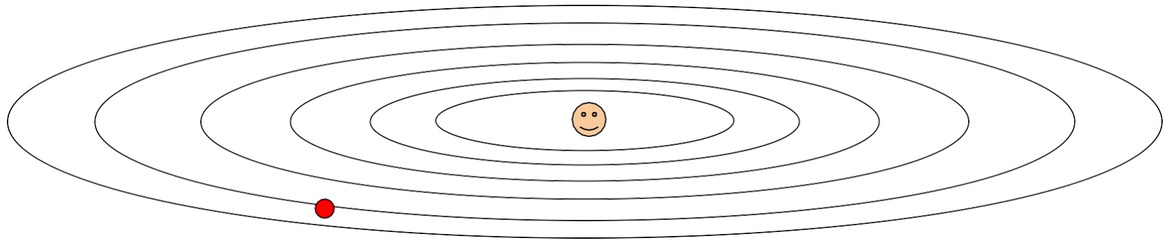
Q: What happens with AdaGrad?

Progress along “steep” directions is damped; progress along “flat” directions is accelerated

If dx is large, np.sqrt(grad_squared) will increased rapidly, Denominator will help decrease the learning rate

AdaGrad

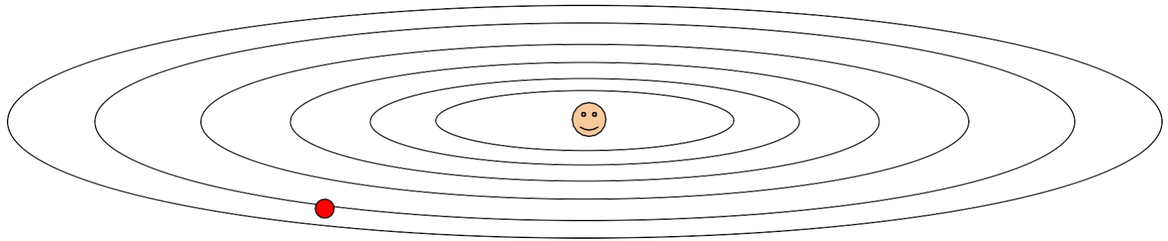
```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



问题2：长时间后步长会发生什么变化?

AdaGrad

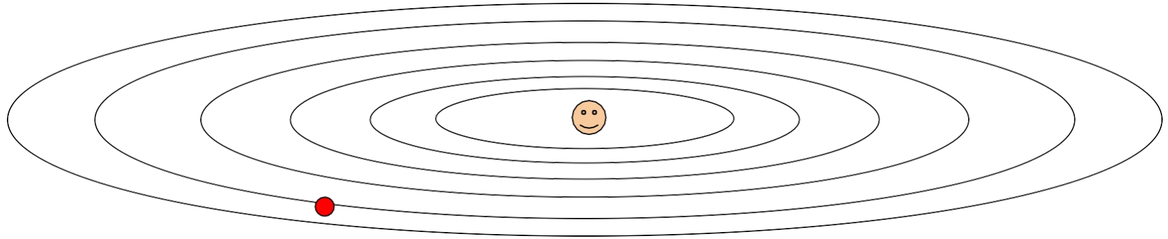
```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Q2: What happens to the step size over long time?

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

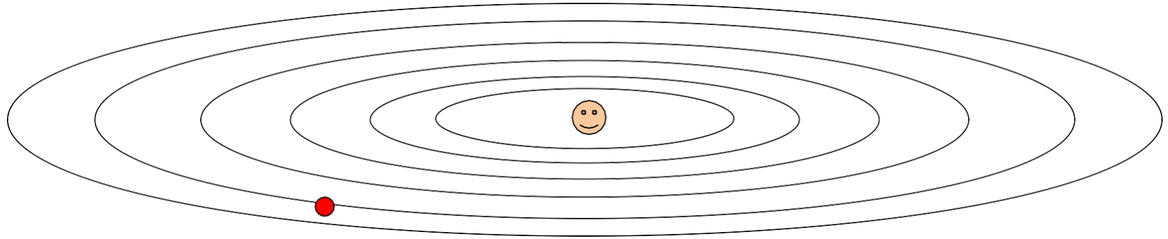


问题2：步长在长时间内会发生什么？衰减至零

o

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Q2: What happens to the step size over long time? Decays to zero

RMSProp

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Tieleman and Hinton, 2012

RMSProp

AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Tieleman and Hinton, 2012

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

动量

AdaGrad / RMSProp

有点像带动量的 RMSProp

Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Momentum

AdaGrad / RMSProp

Sort of like RMSProp with momentum

Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

开始时，second□moment□□。

Kingma 和 Ba，“Adam：一种用于随机优化的方法”，ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

At the beginning, second_moment = 0.

Kingma and Ba, “Adam: A method for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

起初，二阶□矩□□。

这一项非常小



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

At the beginning, second_moment = 0.

This term is very small



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

起初，二阶□矩□□。

我们将迈出非常大的一步。



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

At the beginning, second_moment = 0.

We’ll take a very large step.



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（几乎）

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

动量

AdaGrad / RMSProp

一阶/二阶矩的滑动平均是有偏估计

Kingma 和 Ba，“Adam：一种随机优化方法”，ICLR 2015

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Momentum

AdaGrad / RMSProp

Moving average of first/second moments are biased estimation

Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（完整形式）

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

动量

偏差校正

OdaWrad 3i c SProp

针对一阶和二阶矩估计从零开始这一事实的偏差校正



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

Adam（完整形式）

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

动量

偏差校正

OdaWrad 3i c SProp

针对一阶矩和二阶矩估计从零开始这一事实的偏差校正

Adam 使用 betae □b_□，betat □b_□□□，且 学习□率 □ee-3 或 5e-4 对许多模型来说是一个很好的起点！

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with beta1 = 0.9,
beta2 = 0.999, and learning_rate = 1e-3 or 5e-4
is a great starting point for many models!

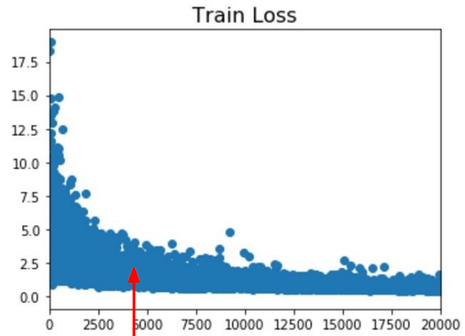
如何确保模型优化良好?

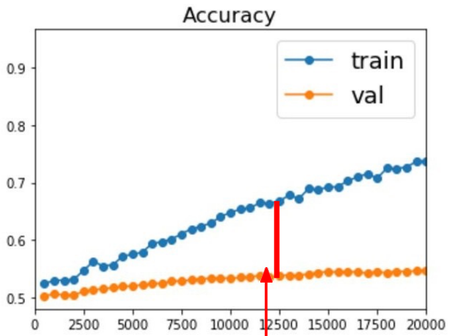
- （上周）非饱和激活函数
- 架构
- 优化器
- 正则化

How do we ensure the model to optimize well?

- (Last week) Non-saturation activation function
- Architecture
- Optimizer
- Regularization

超越训练误差

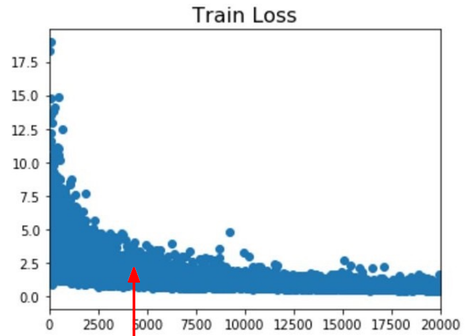


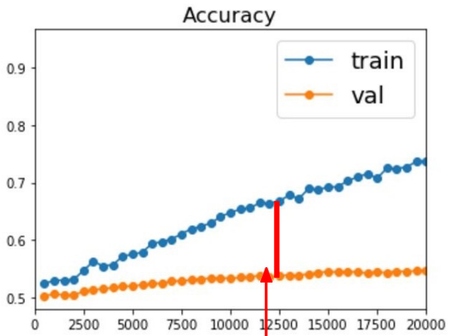


更好的优化算法有助于降低训练损失

但我们真正关心的是新数据上的误差——如何缩小这一差距?

Beyond Training Error

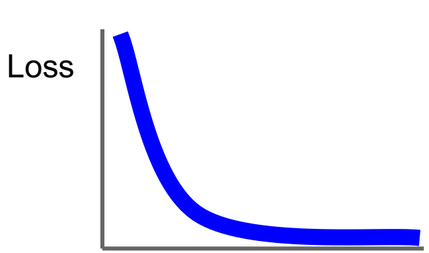


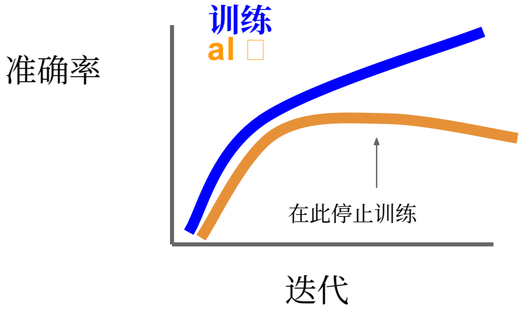


Better optimization algorithms help reduce training loss

But we really care about error on new data - how to reduce the gap?

早停

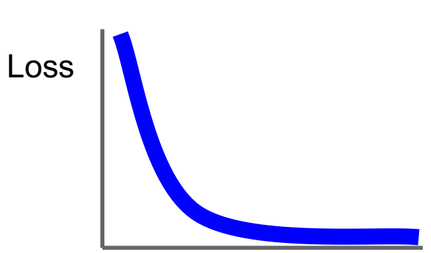


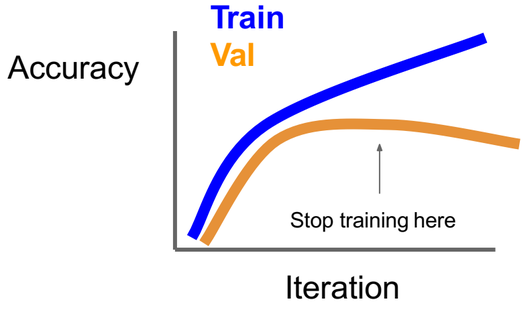


当验证集上的准确率下降时，停止训练模型

或者长时间训练，但始终跟踪在验证集上表现最好的模型快照

Early Stopping



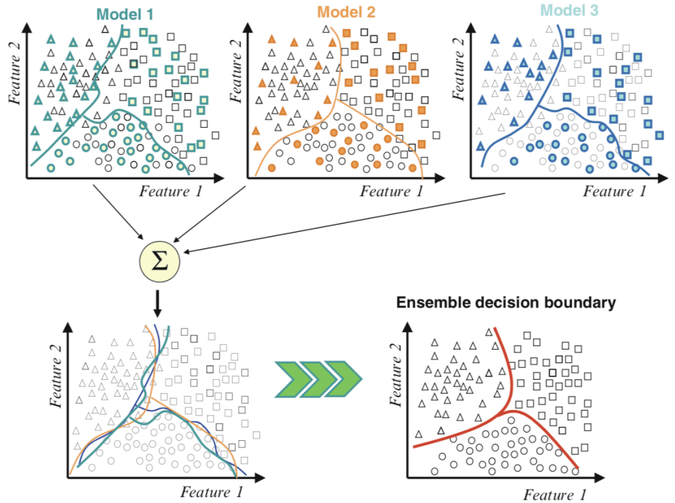


Stop training the model when accuracy on the validation set decreases

Or train for a long time, but always keep track of the model snapshot that worked best on val

模型集成

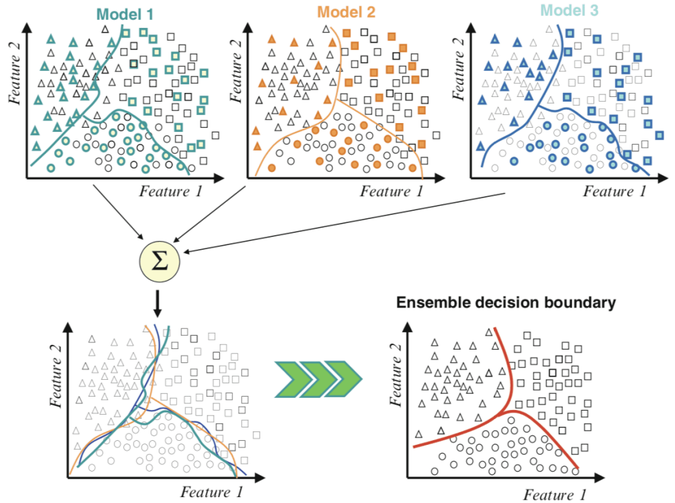
1. 训练多个独立模型 2. 在测试时平均它们的结果（对预测概率分布取平均，然后选择 argmax）



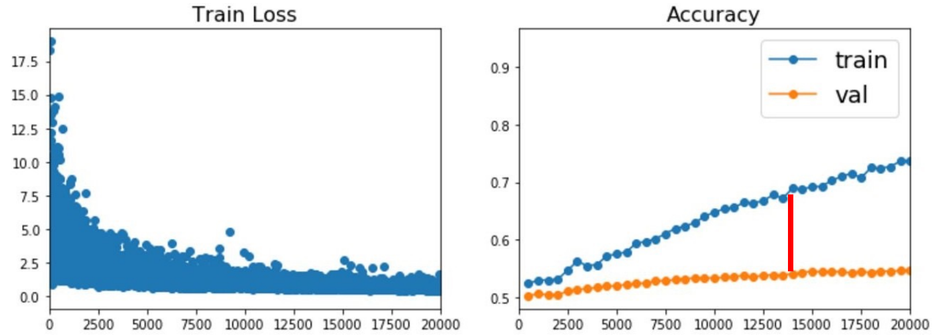
Model Ensembles

1. Train multiple independent models
2. At test time average their results

(Take average of predicted probability distributions, then choose argmax)

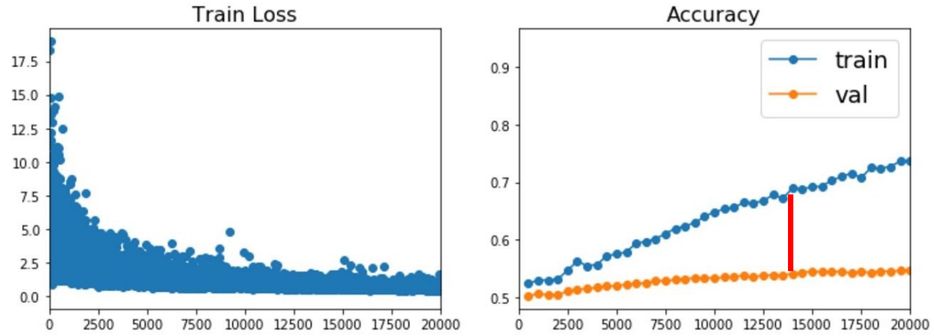


如何提高单模型性能?



正则化

How to improve single-model performance?



Regularization

正则化：向损失中添加项

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

常用做法：

L2 正则化 (权重衰减L1 正$R(W) = \sum_k \sum_l W_{k,l}^2$ (L□□)

L2)

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

Regularization: Add term to loss

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

In common use:

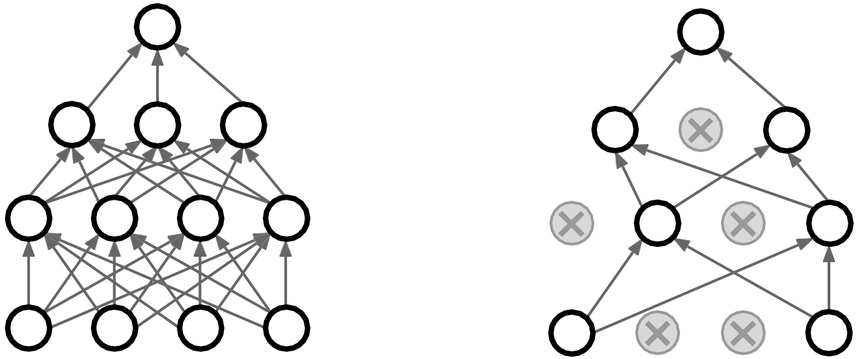
L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$ (Weight decay)

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

正则化：Dropout

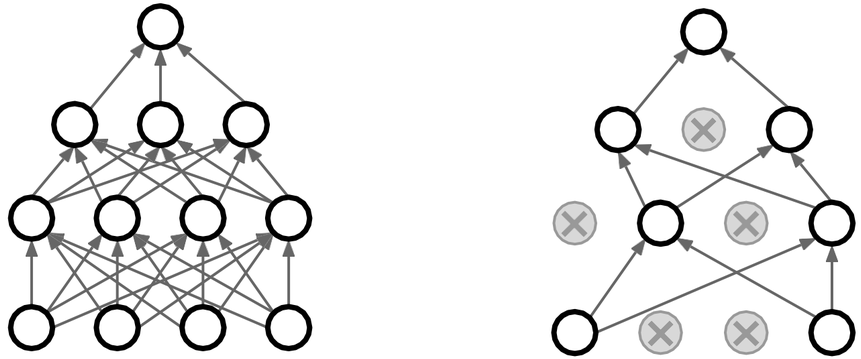
在每次前向传播中，随机将一些神经元置零。丢弃概率是一个超参数；0.5 很常见



Srivastava 等，“Dropout：一种防止神经网络过拟合的简单方法”，JMLR2014

Regularization: Dropout

In each forward pass, randomly set some neurons to zero
Probability of dropping is a hyperparameter; 0.5 is common



Srivastava et al, “Dropout: A simple way to prevent neural networks from overfitting”, JMLR2014

正则化：Dropout

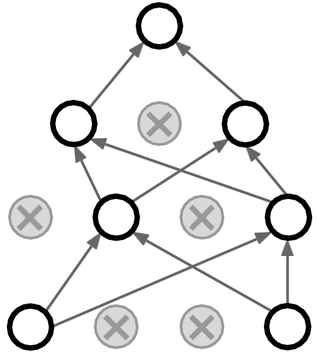
```python
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)
```

使用 dropout 的 3 层网络前向传播示例



Regularization: Dropout

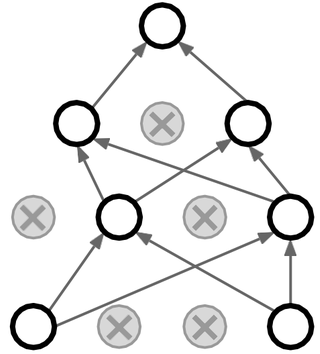
```python
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

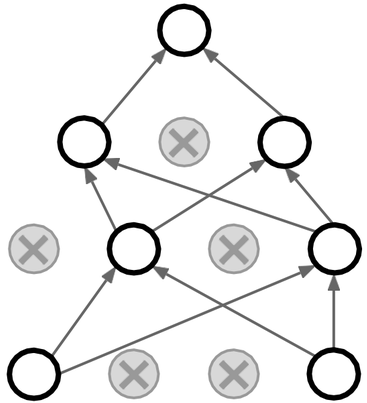
  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)
```

Example forward pass with a 3-layer network using dropout

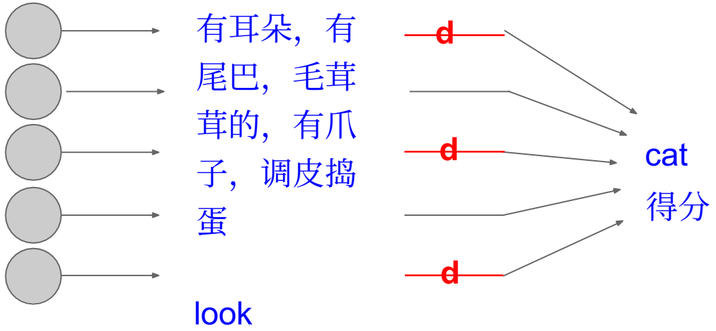


正则化：Dropout

这怎么可能是个好主意?

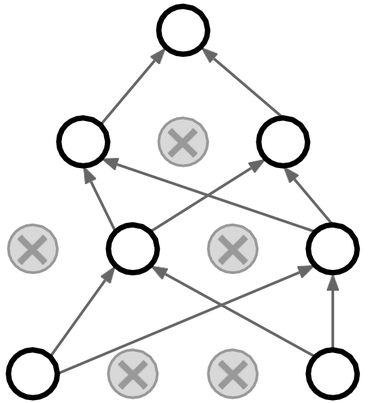


迫使网络具有冗余表示；防止特征共适应



Regularization: Dropout

How can this possibly be a good idea?

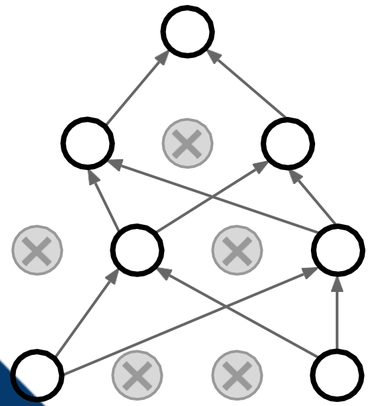


Forces the network to have a redundant representation;
Prevents co-adaptation of features



正则化：随机失活

这怎么会是个好主意呢?



另一种解释：

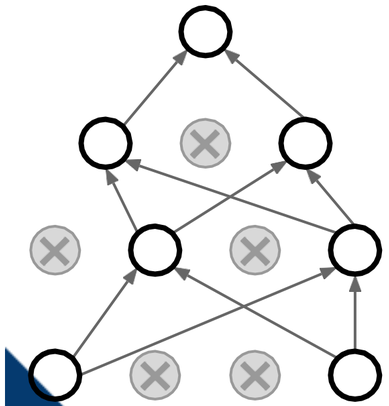
随机失活是在训练一个大型的 **集成** 模型（它们共享参数）。

每个二值掩码都是一个模型

一个具有4096个单元的全连接层有 $\square^{42A8}\ \square\ \square\square^{1233}$ 种可能的掩码！宇宙中只有 $\square\ \square\square^{=2}$ 个原子.....

Regularization: Dropout

How can this possibly be a good idea?



Another interpretation:

Dropout is training a large **ensemble** of models (that share parameters).

Each binary mask is one model

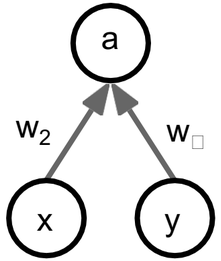
An FC layer with 4096 units has $2^{4096} \sim 10^{1233}$ possible masks!
Only $\sim 10^{82}$ atoms in the universe...

Dropout：测试时

希望近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

考虑单个神经元。

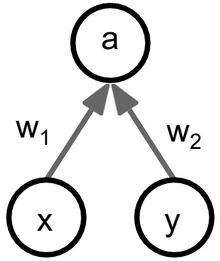


Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

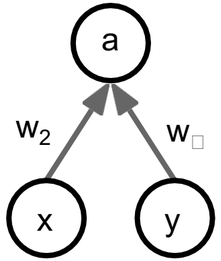
Consider a single neuron.



Dropout：测试阶段

希望近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



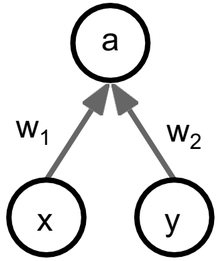
考虑单个神经元。

在测试时，我们有：$E\big[a\big] = w_1 x + w_2 y$

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



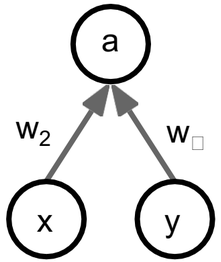
Consider a single neuron.

At test time we have: $E\big[a\big] = w_1 x + w_2 y$

Dropout：测试阶段

想要近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



考虑单个神经元。

在测试阶段，我们有：$E\big[a\big] = w_1 x + w_2 y$

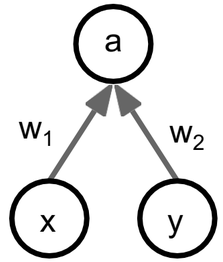
在训练阶段，我们有：

$$\begin{aligned} E[a] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



Consider a single neuron.

At test time we have: $E\big[a\big] = w_1 x + w_2 y$

During training we have:

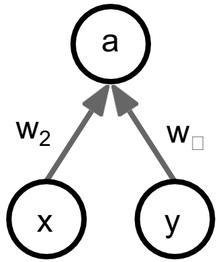
$$\begin{aligned} E[a] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Dropout：测试阶段

想要近似该积分

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

考虑单个神经元。



在测试阶段，我们有：

在训练阶段，我们有：

在测试时，**乘以** dropout 概率

$$E[a] = w_1 x + w_2 y$$

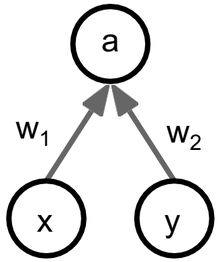
$$\begin{aligned} E[a] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

Consider a single neuron.



At test time we have:

During training we have:

At test time, **multiply** by dropout probability

$$E[a] = w_1 x + w_2 y$$

$$\begin{aligned} E[a] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

随机失活：测试时

```
def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

在测试时，所有神经元始终处于激活状态 =□ 我们必须缩放激活值，使得对于每个神经元：测试时的输出 = 训练时的期望输出



Dropout: Test time

```
def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

At test time all neurons are active always
=> We must scale the activations so that for each neuron:
output at test time = expected output at training time



Dropout

总结

```python
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

训练时丢弃

测试时缩放



Dropout

Summary

```python
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

drop in train time

scale at test time



正则化：一种常见模式

训练：添加某种随机性

$$y = f_W(x, z)$$

测试：对随机性取平均（有时为近似）

$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$



Regularization: A common pattern

Training: Add some kind of randomness

$$y = f_W(x, z)$$

Testing: Average out randomness (sometimes approximate)

$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$



正则化：一种常见模式

训练：加入某种随机性

$$y = f_W(x, z)$$

测试：对随机性取平均（有时近似）

$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$

示例：批量归一化

训练：使用来自随机小批量的统计量进行归一化

测试：使用固定统计量进行归一化

Regularization: A common pattern

Training: Add some kind of randomness

$$y = f_W(x, z)$$

Testing: Average out randomness (sometimes approximate)

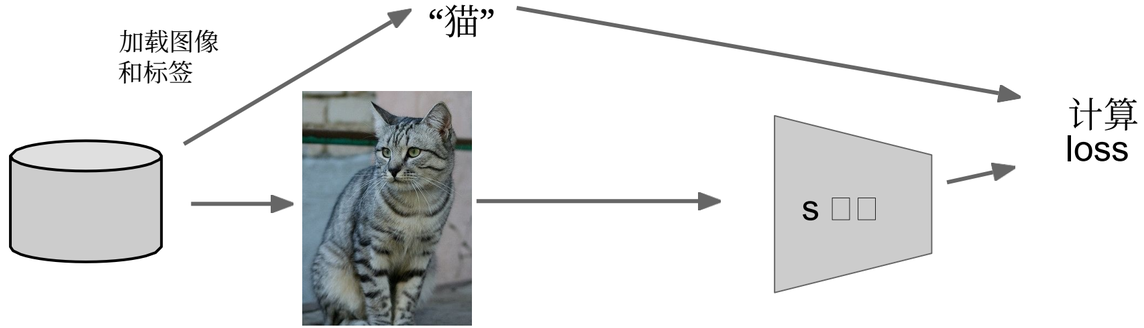
$$y = f(x) = E_z\big[f(x, z)\big] = \int p(z) f(x, z) dz$$

Example: Batch Normalization

Training: Normalize using stats from random minibatches

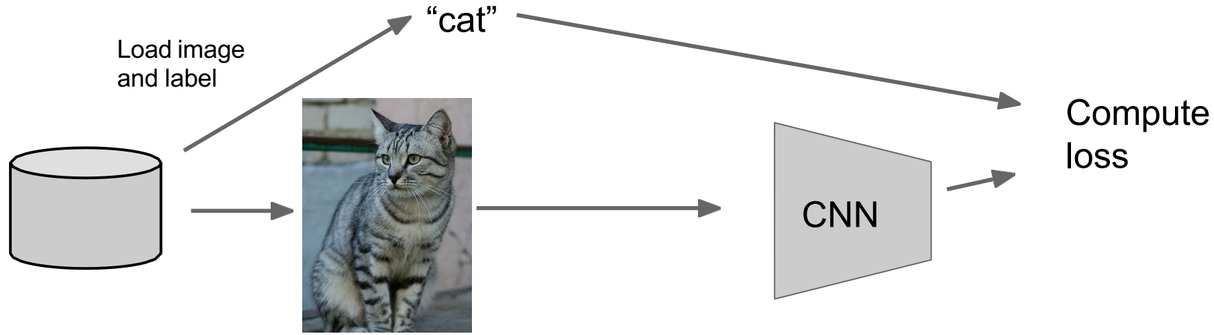
Testing: Use fixed stats to normalize

正则化：数据增强



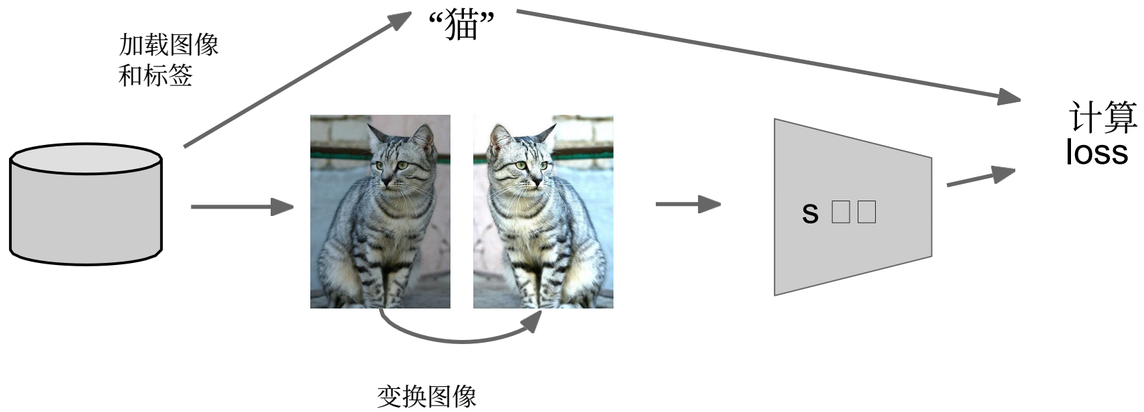
这张图像 由 Nikita 是
在 CC-BY2.0 许可下

Regularization: Data Augmentation

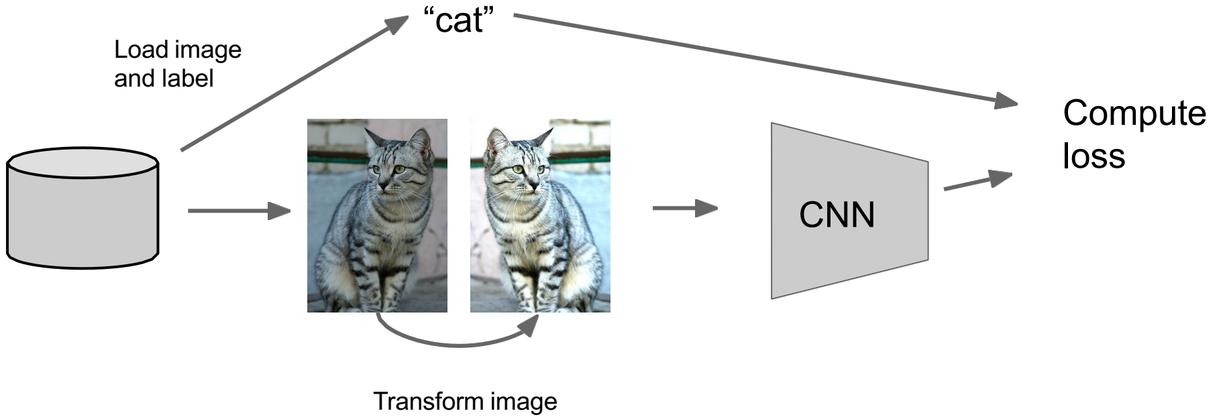


This image by Nikita is
licensed under CC-BY 2.0

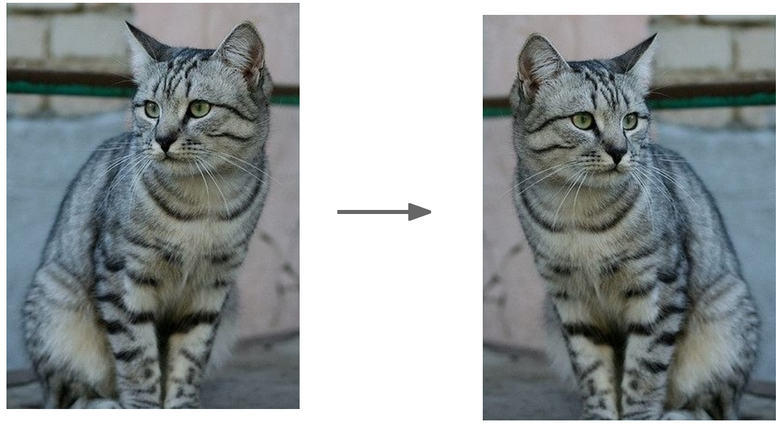
正则化：数据增强



Regularization: Data Augmentation

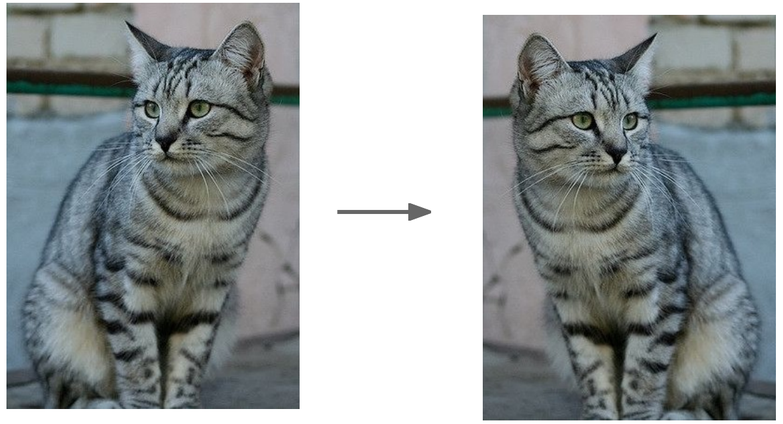


数据增强水平翻转



Data Augmentation

Horizontal Flips



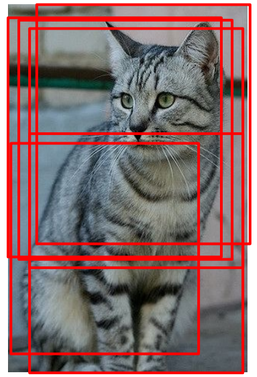
数据增强

随机裁剪与尺度

训练：采样随机裁剪 / 尺度ResNet：

1 □2CN0=[4□. 在范围内
2

随机选取 L。调整训练图像大小，短边 l L。3. 随机采样 224 x 224 图像块



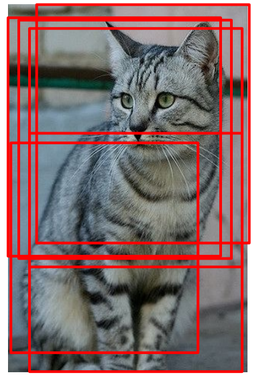
Data Augmentation

Random crops and scales

Training: sample random crops / scales

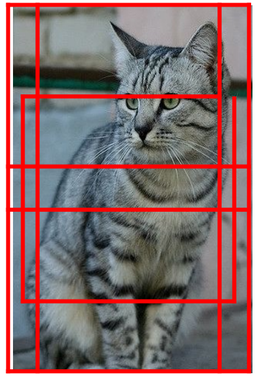
ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch



数据增强

随机裁剪与缩放



训练：采样随机裁剪 / 尺度ResNet：

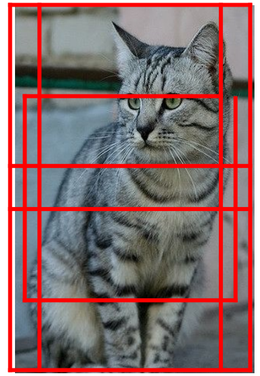
1 2CN0=[4□. 在范围 内随机选取 L。将训练图像缩放，短边为 l L。3. 随机采样 224 x 224 的图像块

测试：对一组固定的裁剪取平均ResNet：

1. 在5种尺度上调整图像大小：{224, 256, 384, 480, 640} 2. 对于每个尺寸，使用10个224 x 224裁剪区域：4个角，中心，，翻转 ips

Data Augmentation

Random crops and scales



Training: sample random crops / scales

ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch

Testing: average a fixed set of crops

ResNet:

1. Resize image at 5 scales: {224, 256, 384, 480, 640}
2. For each size, use 10 224 x 224 crops: 4 corners + center, + flips

数据增强

颜色抖动

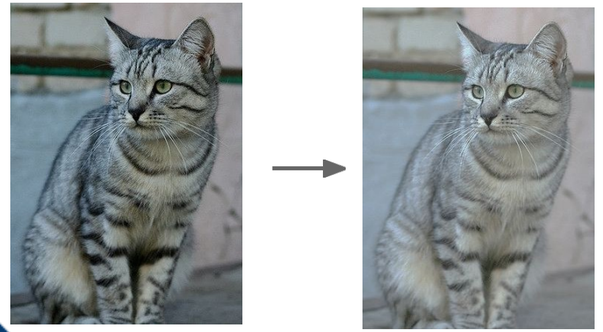
简单：随机化对比度和亮度



Data Augmentation

Color Jitter

Simple: Randomize contrast and brightness



数据增强

为你的问题发挥创意！

数据增强的示例：- 平移 - 旋转 - 拉伸 - 剪切，- 镜头畸变，...（尽情发挥）



Data Augmentation

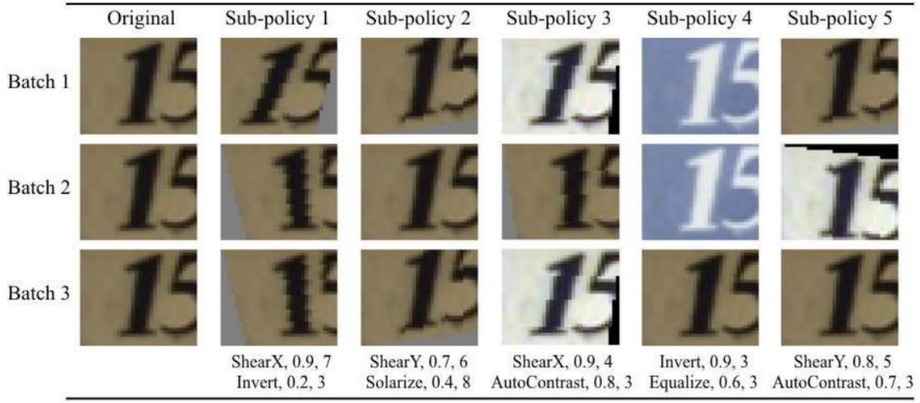
Get creative for your problem!

Examples of data augmentations:

- translation
- rotation
- stretching
- shearing,
- lens distortions, … (go crazy)

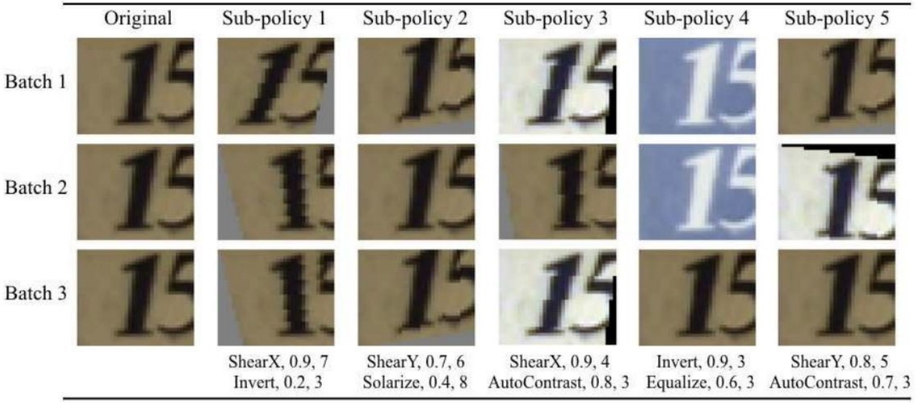


自动数据增强



Cubuk et al., “AutoAugment: Learning Augmentation Strategies from Data”, CVPR 2019

Automatic Data Augmentation



Cubuk et al., “AutoAugment: Learning Augmentation Strategies from Data”, CVPR 2019

迁移学习

Transfer learning

“如果你想训练/使用 CNNs，就需要大量数据”

“You need a lot of a data if you want to train/use CNNs”

“如果你想的话，[illegible]需要大量数据来训练/使用 CNNs”

“You need a lot of [illegible]ata if you want to train/use CNNs”

使用 CNNs 的迁移学习

Donahue et al，“DeCAF：面向通用视觉识别的深度卷积激活特征”，ICML 2014；Razavian et al，“CNN 特征开箱即用：一个惊人的识别基线”，CVPR Workshops 2014

1. 在 ImageNet 上训练



Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014
Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

1. Train on Imagenet



使用 CNNs 的迁移学习

Donahue 等，“DeCAF：一种面向通用视觉识别的深度卷积激活特征”，ICML 2014 Razavian 等，“CNN 特征开箱即用：一个令人惊叹的识别基线”，CVPR Workshops 2014

1. 在 ImageNet 上训练



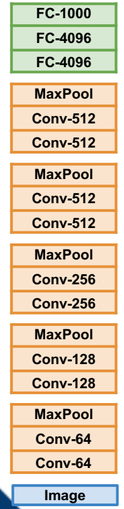
2. 小数据集（C 类）



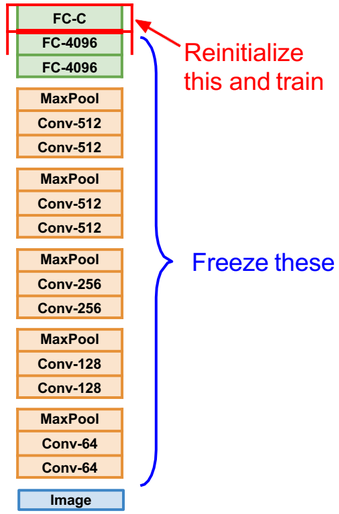
Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014
Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

1. Train on Imagenet



2. Small Dataset (C classes)



使用 CNNs 的迁移学习

Donahue 等，“DeCAF：一种用于通用视觉识别的深度卷积激活特征”，ICML 2014 Razavian 等，“现成的 CNN 特征：一个惊人的识别基线”，CVPR Workshops 2014

1. 在 ImageNet 上训练



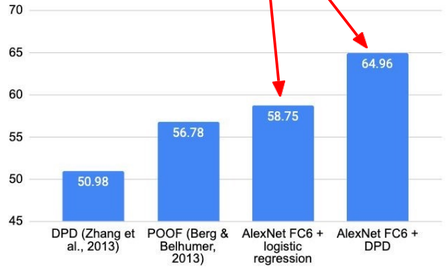
2. 小数据集（C 类）



重新初始化
并对此进行训练

冻结这些

基于 AlexNet 微调

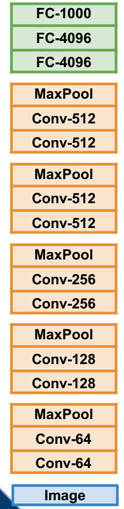


Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

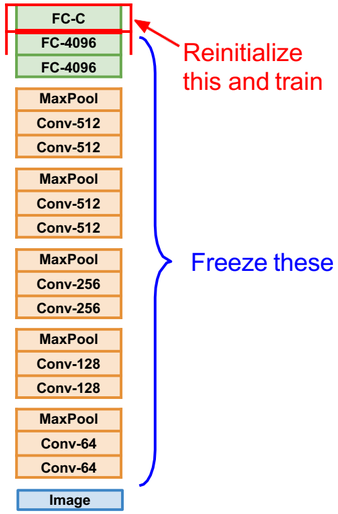
Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014
Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

1. Train on Imagenet



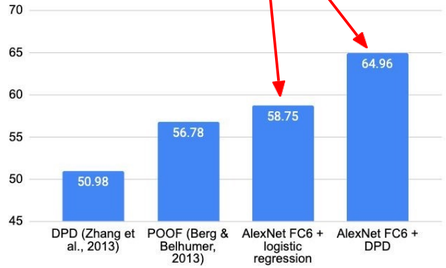
2. Small Dataset (C classes)



Reinitialize
this and train

Freeze these

Finetuned from AlexNet

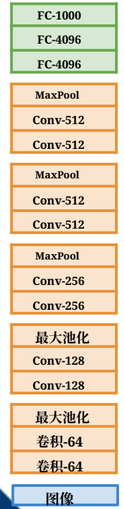


Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

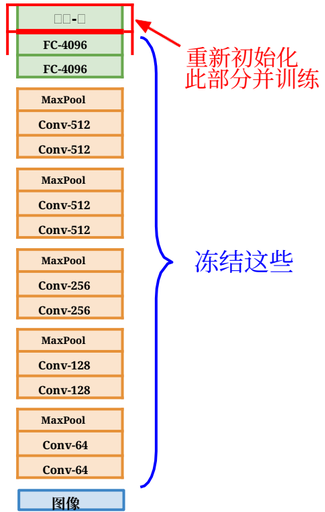
基于 CNNs 的迁移学习

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline f or Recognition”, CVPR Workshops 2014

1. 在 ImageNet 上训练



2. 小数据集（C 类）



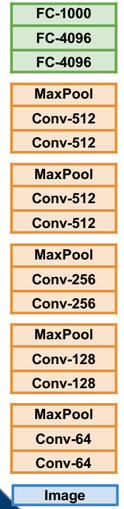
3. 更大的数据集



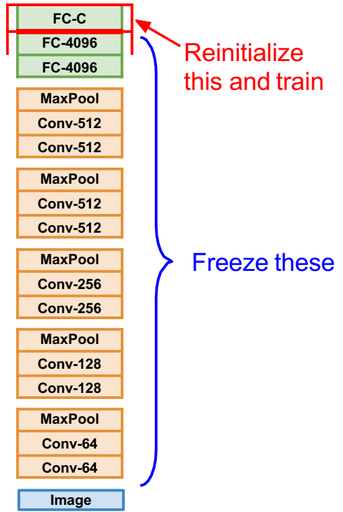
Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

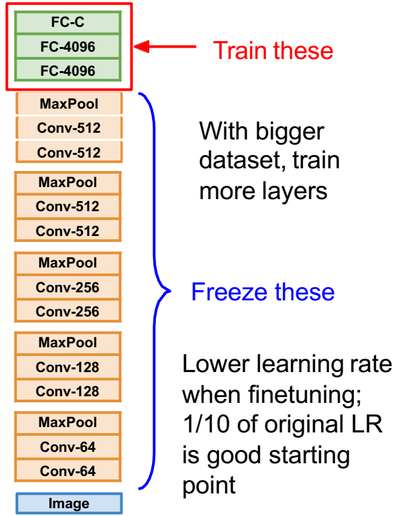
1. Train on Imagenet

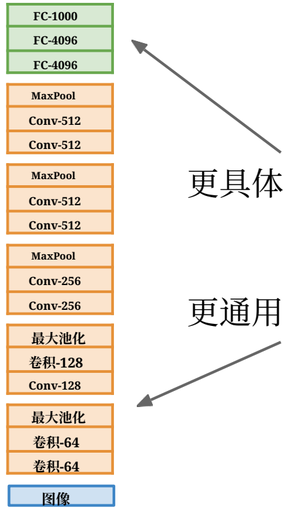


2. Small Dataset (C classes)



3. Bigger dataset

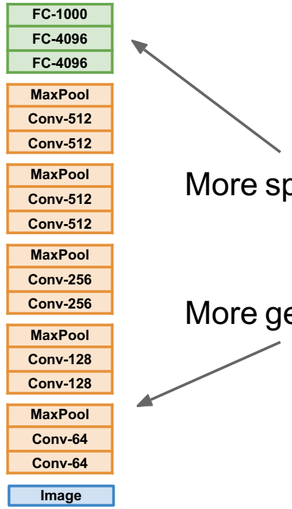




更具体

更通用

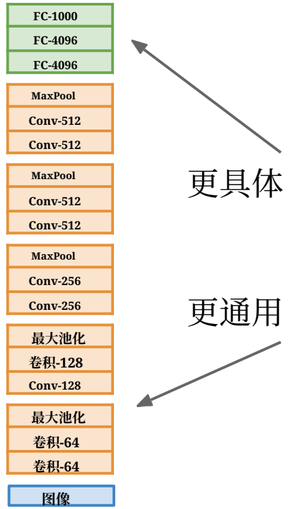
	非常相似 数据集	非常不同 数据集
非常少的数据	N	N
相当多的 □□	N	N



More specific

More generic

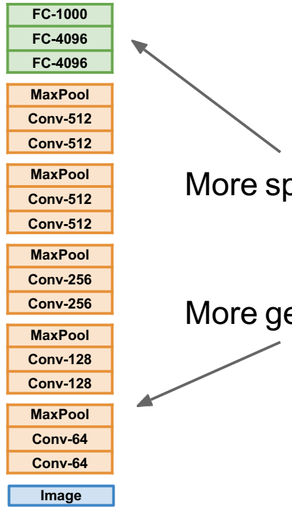
	very similar dataset	very different dataset
very little data	?	?
quite a lot of data	?	?



更具体

更通用

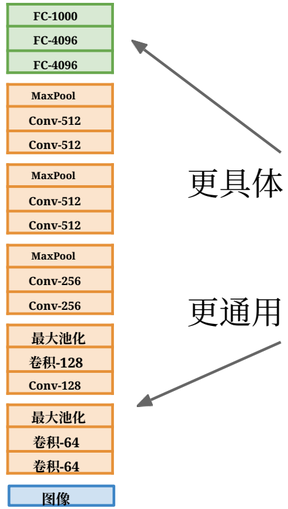
	非常相似数据集	**非常不同数据集**
数据非常少	使用线性分类器作用于顶层	N
相当多 □ □	微调少数几层	N



More specific

More generic

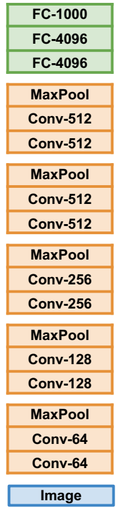
	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	?
quite a lot of data	Finetune a few layers	?



更具体

更通用

	非常相似数据集	**非常不同数据集**
数据非常少	使用线性分类器作用于顶层	你就成功了 麻烦.....试试线性分类器来自不同阶段
相当多的 □I □I	微调一个少量层	微调更多层



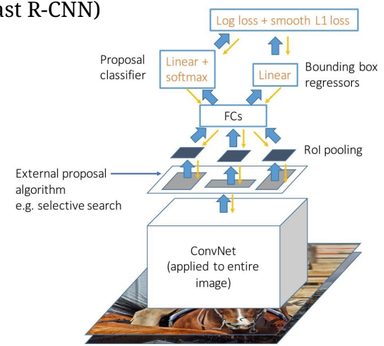
More specific

More generic

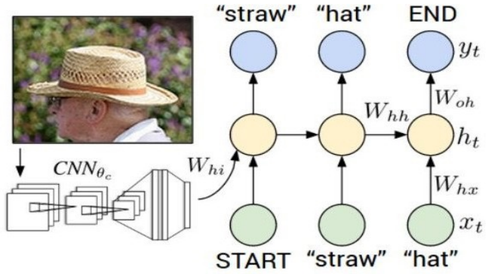
	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

使用CNNs的迁移学习非常普遍... (这是常态，而非例外)

目标检测
(Fast R-CNN)



图像描述：CNN + RNN



Girshick, "Fast R-CNN", ICCV 2015 图版权 Ross Girshick, 2015。经许可复制。

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015 图版权 IEEE, 2015。为教育目的复制。

Transfer learning with CNNs is pervasive...
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)

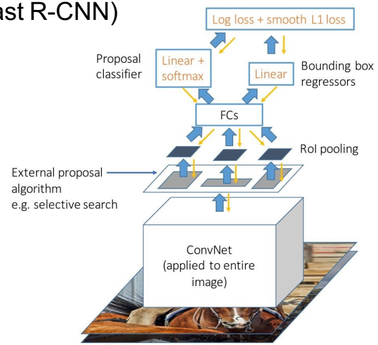
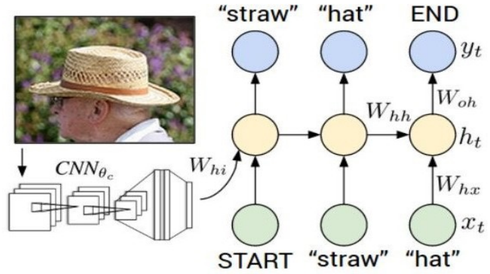


Image Captioning: CNN + RNN

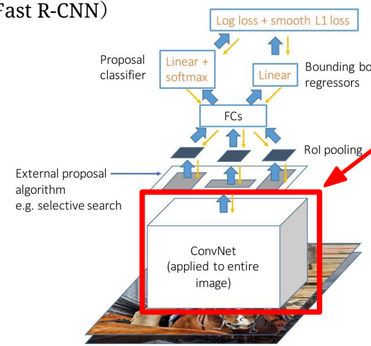


Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

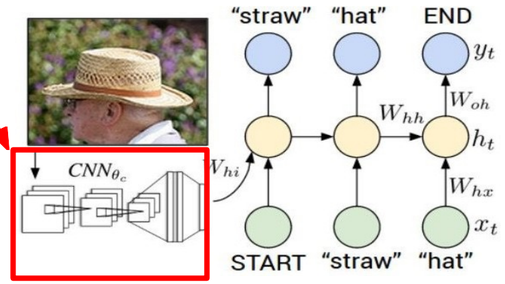
基于 CNNs 的迁移学习已无处不在……（这是常态，而非例外）

目标检测
（Fast R-CNN）



CNN 预训练
在 ImageNet 上

图像描述生成：CNN + RNN

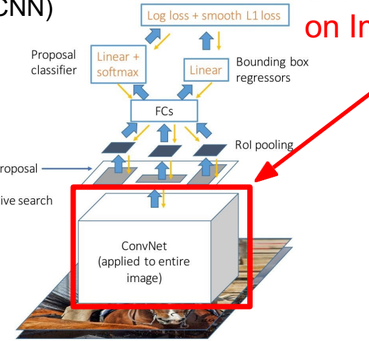


Girshick，“Fast R-CNN”，ICCV 2015 图版权归 Ross Girshick 所有，2015。经许可转载。

Karpathy 和 Fei-Fei，“Deep Visual-Semantic Alignments for Generating Image Descriptions”，CVPR 2015 图版权归 IEEE 所有，2015。出于教育目的复制。

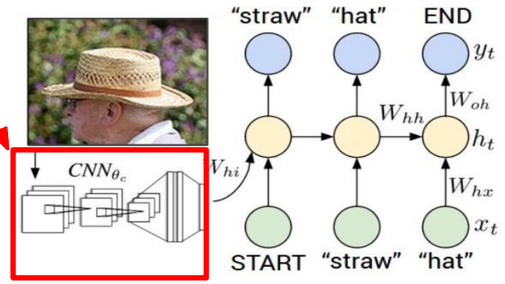
Transfer learning with CNNs is pervasive…
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained
on ImageNet

Image Captioning: CNN + RNN

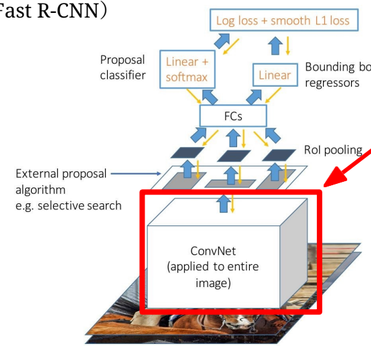


Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

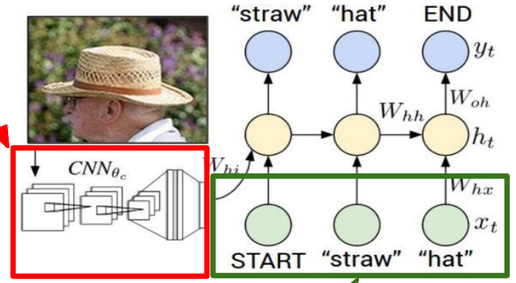
使用 CNNs 的迁移学习十分普遍……（这是常态，而非例外）

目标检测
（Fast R-CNN）



预训练的 CNN
在 ImageNet 上

图像描述：CNN + RNN



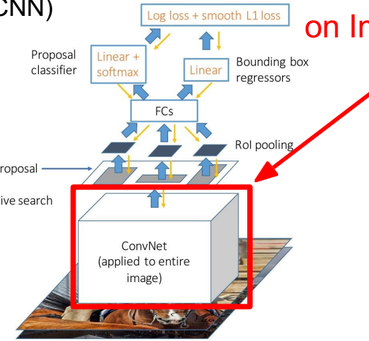
预训练的词向量
with word2vec

Girshick, "Fast R-CNN", ICCV 2015
图版权归 Ross Girshick 所有，2015 年。经许可复制。

Karpathy 和 Fei-Fei，"深度视觉-语义对齐用于 Generating Image Descriptions", CVPR 2015
图版权归 IEEE 所有，2015。为教育目的复制。

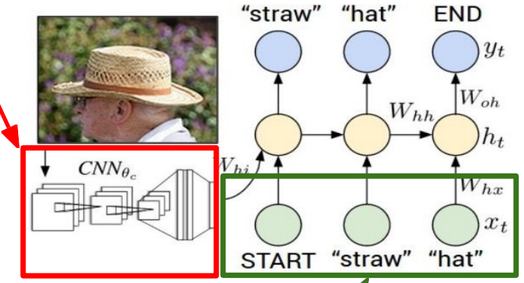
Transfer learning with CNNs is pervasive…
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained
on ImageNet

Image Captioning: CNN + RNN



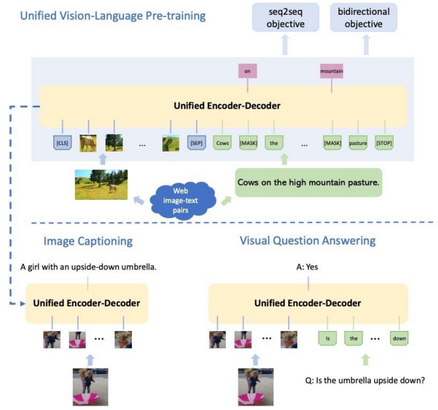
Word vectors pretrained
with word2vec

Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

基于 CNNs 的迁移学习非常普遍……（这是常态，而非例外）

视觉问答的一个示例



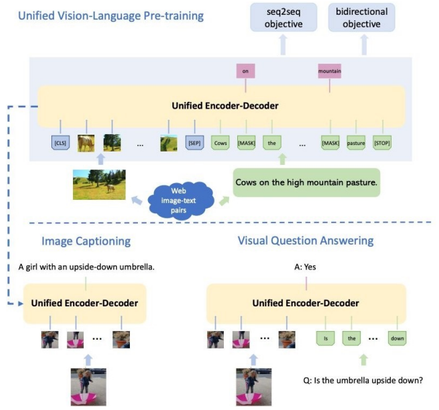
1. 在ImageNet 上训练 CNN 2. 针对目标检测微调 (1) 3. 在大量文本上训练BERT 语言模型 4. 结合 (2) 和 (3)，训练联合图像 / 语言建模 5. 针对图像描述、视觉问答等微调 (4)。

Krishna 等，“Visual Genome：使用众包密集图像标注连接语言与视觉” IJCV 2017 Devlin 等，“BERT：用于语言理解的深度双向 Transformer 预训练” ArXiv 2018e

Zhou 等，“面向图像描述与 VQA 的统一视觉-语言预训练” CVPR 2020
图像权 Luowei Zhou，2020。经许可转载。

Transfer learning with CNNs is pervasive…
(it’s the norm, not an exception)

An example of Visual Question Answering



1. Train CNN on ImageNet
2. Fine-Tune (1) for object detection
3. Train BERT language model on lots of text
4. Combine(2) and (3), train for joint image / language modeling
5. Fine-tune (4) for image captioning, visual question answering, etc.

Krishna et al, “Visual genome: Connecting language and vision using crowdsourced dense image annotations” IJCV 2017
Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” ArXiv 2018e

Zhou et al, “Unified Vision-Language Pre-Training for Image Captioning and VQA” CVPR 2020
Figure copyright Luowei Zhou, 2020. Reproduced with permission.

匿名课堂反馈

Q&A

Anonymous
Class Feedback

Q&A